

kayfabe — the design, the plan, and what we know
Stated as decided, 2026-09-20. Written to be argued with from scratch.

branch w749-fable-legb

Contents

How to read this	4
The markers	4
What this design is for	4
Part 1 — The design	5
1. What this is	5
2. What varies — the seven axes	5
3. The shape	6
4. Two privilege boundaries	6
5. The vCPU trap path	7
5.1 Two structures, because one cannot do both jobs	8
5.2 The token's four states	8
5.3 The wakeup word	8
5.4 The privileged ring	9
5.5 The shadow	9
5.6 Doorbells versus registers, across the boundary	10
5.7 The doorbell page itself	11
6. Memory and addresses	11
6.1 $\square\square\square$ The guest's BAR tables are metadata. Nothing but us ever reads them.	12
6.1 BAR1 and BAR2 are split between us and the guest	13
7. Channels	14
8. Completions and interrupts	14
8.1 $\Rightarrow$ We forward the host's edge, and never decode the request method	15
9. The host side	15
9.2 $\square$ Per-VM caps, because a shared host GPU is a shared resource	16
9.4 $\square$ Two host-platform prerequisites, stated because they are not ours to fix	16
9.1 Multi-GPU: what is per device and what is per process	16
9.2 Platform decisions	17
10. What is deleted, and why	17
Part 2 — The plan	18
0. The three rules that shape everything below	18
1. The crate model — \$10.4 discharged	18
2. The phases	19
$\oslash\oslash\oslash$ P-1 — THE QEMU BQL BYPASS. Item zero, and without it nothing below is measurable.	19
P0 — The harness, before any product code	19
P1 — kf-trap: the vCPU path	19
P2 — GSP boot to INIT_DONE	19
P3 — the RPC and object plane	20
P4 — memory and the VA plane	20
P5 — channels and the doorbell	20
P6 — the data plane, and the defect that is waiting there	20
P7 — compute	20
P4.5 — reset, teardown and re-init. $\oslash$ Not optional, and it blocks the suite being trusted.	20
P5.5 — error paths: RC, host failure, and what the guest is told	20
P8 — deletions	20
3. Ordering constraints — where getting it wrong forces a rewrite	21
4. Is 50 k plausible? — honestly	21
4.1 $\oslash\oslash\oslash$ An adversarial re-count says 50 k is wrong. I think it is right, and here is the gap.	21
5. What this plan does NOT do, said plainly	22

6. ∅ The three things most likely to be underestimated	22
Part 3 — The surface, at constant level	23
0. The four planes, and why they are different shapes	23
1. GSP RPC — the plane we answer as the firmware	23
1.1 What this plane is, for a reader who does not know NVIDIA's constants	23
1.2 The structures on the wire	23
1.3 The functions	24
1.4 □ The reply is always built from local state	27
2. RM objects and control commands — the plane that rides inside the RPC	27
2.1 What this plane is	27
2.2 The object classes we model	27
2.3 Control commands	29
2.4 □□□□ The synchronous verbs — the list the trap path is judged against	30
□□□ The two status-only verdicts hold — and they are the result worth keeping	31
∅∅∅ The “highest-value gap” was not a gap — retracted	31
□ The UVM gap closes — in the negative	31
3. BAR0 MMIO — reads from DRAM, writes to the vCPU	32
3.1 The asymmetry, and why it exists	32
3.2 The shadow — two structures, and they are not the same thing	32
3.3 The trapped-write register table	32
3.4 □□□ WPR2 is the worked example of two axes crossing in one register	35
3.5 Whole apertures, not registers	35
4. Channels and pushbuffers — the plane unprivileged guest code drives	36
4.1 What a channel is, for a reader who does not know NVIDIA's constants	36
4.2 The three kinds — and an honest statement of what is built	36
4.3 The GPFIFO entry — 8 bytes	37
4.4 The method header — one dword	38
4.5 Every pushbuffer method we recognise	38
4.6 What happens when each kind's doorbell rings	40
5. □ Where this inventory is knowingly incomplete	40
5.1 Gaps in what we implement	40
5.2 Questions a survey could not answer	41
5.3 ∅ Axis coverage — the honest statement	42
5.4 What would close these	42
5.5 ∅∅∅ What the review of THIS document found, kept as the record	42
Part 4 — What the oracles tell us	43
0. ∅∅∅ THE HEADLINE REFUTES WHAT I TOLD THE OWNER YESTERDAY	43
1. □□□□ The reading technique — the VF HAL arm is the surviving proxy for monolithic	43
2. The four artifacts worth having	43
2.1 □□□□ inc/kernel/gpu/gpu_child_list.h — the complete monolithic object model	43
2.2 □□□□ ROUTE_TO_PHYSICAL — a machine-readable manifest of the CPU/GSP boundary	44
2.3 □□□□ 42 orphaned monolithic bodies — defined once, referenced nowhere	44
2.4 □□□ 163 readable if (RMCFG_FEATURE_PLATFORM_GSP) blocks — the firmware's own code	44
3. □□□□ The finding that lands on OUR CURRENT DESIGN: BAR1/BAR2 are SPLIT	44
3.1 Fault-buffer ownership is the sharpest divergence	45
4. Pre-Turing — constants are thin, implementations are LIVE	45
4.1 □ Version rule: use 580 for pre-Turing UVM, 610 for everything else	45
5. Windows residue beyond what Part 3 already had	46
6. = What this changes	47
Part 5 — The guest-OS axis	48
0. □□□ Why this is answerable at all — and exactly where the oracle stops	48
1. ∅∅∅ THE HEADLINE WAS WRONG — GSP is default-ON for Turing+, on every OS	48
∅ And the registry key does not mean what the community advice implies	49
∅∅∅ AND THE RETRACTION ABOVE OVER-CORRECTED — the struct IS defined, and it changes the answer	49
∅∅∅ How I got the retraction wrong — I read a build with the branch compiled out	49
□ So what IS established, stated at the right strength	49
1.1 ∅∅∅ What the public record actually contains — searched w821	50
∅∅∅ And the precise reason the claim is untested: it has no known-positive	50
□□□ SETTLED w821 — the owner measured it, and the known-positive was already on the record	51
□ How far that generalises — three different strengths, and they must not be merged	51

□□□ How the survey got it wrong, and it is this tree's own documented failure	52
□□ What replaces it, and it matters on every OS	52
2. Under WDDM the OS owns the page tables — and we never see the writes	52
3. TDR — a hard, OS-enforced 2-second deadline	52
4. Four more that matter, briefly	53
5. □□□ The escape hatch — TCC, and it collapses most of the table	53
6. □ What is the SAME — this bounds the work, and it is a lot	53
7. □□□ A free gift — detecting a Windows guest from the FIRST RPC	54
8. □ A correction to this project's own framing	54
9. What could NOT be determined, and the first measurements to take	54
10. □ Could we support a GSP-DISABLED guest? — nouveau measured, w821	55
10.1 □ nouveau IS the right oracle, structurally	55
10.2 The measurement	55
10.3 ∅∅∅ The real cost is not lines — it is losing the oracle	56
10.4 What it would buy, stated fairly	56
10.4a ∅∅∅ THE ORACLE OBJECTION IS ANSWERED — mmioTRACE, and it is still in mainline	56
10.5 [PROPOSE] A spike, not a commitment — and the decisive hour	57
10.6 □ OWNER RULING, w821 — the option stays open, and nouveau is the standing oracle	57
10.7 The no-GSP plane's boundaries and sources — owner rulings, w821	57
Part 6 — What is still open	60
1. Open, and cheap to close	60
2. Open, and genuinely undecided	60
3. Deliberately not decided — deferred with a reason	61
4. ∅ Decided the other way — what we give up, and why it is worth it	61
5. □ The three things most likely to be underestimated	62

How to read this

This is a **decision document, not a record of how the decisions were reached**. Everything here is stated as it ended up, with the reasoning that supports it. Where an earlier draft said something different, that is in git and not in these pages.

part	what it is	attack it if...
1. The design	what kayfabe is, and how it works	you think a mechanism is wrong, or a simpler one exists
2. The plan	what gets built, in what order, and what proves each step	you think the sequence is wrong or a phase is mis-sized
3. The surface	every RPC, class, control, register and pushbuffer method we touch, by name and hex	you think something is missing, or we do the wrong thing with it
4. What the oracles tell us	what NVIDIA's open driver and nouveau establish, and where each stops	you think a claim is unsupported, or a source says otherwise
5. Open questions	what is genuinely undecided, and what each would cost	you think something here is actually decided, or something elsewhere should be here

The markers

marker	meaning
□	a load-bearing fact — the argument rests on it
∅	a refusal, a deletion, or a thing that is not true
□	a trap, or a claim whose evidence is weaker than it looks
[MEASURED]	a number from a real run
[ogkm] / [nouveau]	read out of that source, with file:line
[PROPOSE]	a design opinion, stated as one

□ **Repetition marks how much something matters, not whether it is settled.** A settled question says so in words.

What this design is for

A guest runs a **stock, unpatched NVIDIA driver** against an emulated GPU and a firmware processor that does not exist. Real compute reaches a real host GPU from an **unprivileged** host process.

The cut line is **below** D3DKMT and below `/dev/nvidia*` — lower than any shipping product cuts. That is the whole difficulty and the whole point: an ioctl allowlist filters *what* is called, while modelling the device forces us to know what it *means*.

□ **The value proposition is guest-internal isolation.** A sandboxed process inside the guest must not be able to reach guest root through us. That single requirement shapes the trap path, the doorbell plane, and most of what §5 of Part 1 says.

Part 1 — The design

STATUS: LIVE, 2026-09-20 (w821). This states the design as decided. It is written to be argued with from scratch: no correction history, no record of what earlier drafts said. That history is in git, and in the superseded documents this one replaces.

1. What this is

A guest runs a **stock, unpatched NVIDIA driver**. It sees an emulated GPU and a GSP firmware processor that does not exist. We are that firmware. The guest's real compute reaches a real host GPU, driven by an **unprivileged** host process.

Two properties make it worth building rather than forwarding ioctls like `nvproxy` or `WSL2`:

- **□ The guest driver is unmodified and unaware.** No guest kernel module, no paravirtual ABI, no host-side driver patch. The cut line is **below** `D3DKMT` and below `/dev/nvidia*` — where no shipping product cuts.
- **□□□ Modelling the device forces us to know what things mean.** An allowlist filters *what* is called; we must know *why*. That is what lets us refuse a flag whose hazard lives in RM's interpretation of a bit — including one NVIDIA's own source labels *"This is a bug"*.

Scope. GSP-capable parts, **Turing and newer**, because below Turing there is no GSP to be. Linux and Windows guests; Linux host. QEMU first, VMM-agnostic core. Multi-GPU in the data model from the start, not in the first milestone.

2. What varies — the seven axes

Nearly nothing here may be a literal in source, because seven things move underneath it.

tag	axis	posture	absorbed by
K	guest kernel	all major versions	□ not struct layouts — DMA addressing : a vIOMMU guest hands us IOVAs, not GPAs
Dg	guest driver version	all major versions	generated per-version descriptors, never if version ==
Dh	host driver version	all major, ∅ decoupled from Dg	we author every host call; our host verbs take no guest flag word
A	GPU architecture	Turing+	a format-family descriptor — four families span Turing→Blackwell
die	die within an arch	any	derived per die, maintained per family
V	VMM	● a version floor is legitimate here only	the core is VMM-agnostic; one shim knows QEMU
OS	guest OS	Linux and Windows	detect, never assume

Three rules follow. ∅ No literal in a comparison. ∅ No C parsed with regex — generate from headers with a real parser, and from `rnndb`'s XML where only `nouveau` has the data. ∅ No guest-supplied flag word forwarded into a host call; that is a `Dg↔Dh` coupling in disguise.

□ **The worked example that proves the descriptor approach:** the GSP message-element header is **48 bytes** at driver 580 (`authTag/aad/checkSum@32/seqNum@36/elemCount@40`) and **16 bytes** at 610 (`mctpHeader/nvdmHeader/checkSum@8/seqNum@12`). Size and every offset moved, and a field disappeared. It is absorbed as data. □ And it marks the limit: a version break can also move **derivation** and **validation**, which a layout descriptor cannot hold.

□ **Axes interact.** WPR2 moved on Blackwell to `0x0088A824` and still answers the legacy `0x1FA824` for 580-series guests — the **die** moved it, the **driver version** decides which address is in play. A per-die table alone answers the wrong address.

3. The shape

One process: the VMM with the kayfabe core linked in. No isolate children, no IPC, no sandbox plane of our own — sandbox the whole hypervisor process instead.

thread	count	job
vCPU	guest's	traps only. Never blocks, never waits on us
worker	$N \leq 255$	the doorbell plane: scan, claim a token, run the channel's work
register drainer	1	drains the privileged ring in order . One thread, because an ordered ring drained by many is not ordered
VA manager	1	all mmap/munmap — page-table diffs, BAR windows, channel mappings

□ The VA manager is a thread rather than a lock so that “*who may map*” is a structural answer. □ Workers are capped at **255** structurally: the poller count is 8 bits, and a wrap would read zero while workers are polling — the exact lost wakeup the protocol exists to prevent.

The latency budget is asymmetric, and it decides most design questions. Inside an MMIO trap: microseconds, and **the tail is the statistic**. Everywhere behind it: milliseconds. ⇒ Spending a thread to keep the trap lock-free is a good trade; moving work into the trap to save a context switch behind it is a defect however good its average looks.

□ **The VMM must be patched for this to be true, and the patch is six lines.** QEMU takes its global lock unconditionally on every MMIO dispatch, and the per-region opt-out that used to exist was removed upstream. □ But the KVM exit site itself is **already outside** that lock — it is re-taken one level down, in the dispatch helper. ⇒ The fix is to **restore the opt-out as a flag on the region's operations**, set only on our trap regions, not to write an exit-site dispatcher.

◻◻◻ **And the patch must disable one more thing, or it creates the very hole it exists to avoid.** The VMM guards every device's I/O dispatch with a **plain, non-atomic boolean** — “*do not allow more than one simultaneous access to a device's I/O regions*”. Under the global lock that only trips on true re-entrancy. **Without it, two vCPUs writing our device concurrently race on that boolean, and the loser's write returns an access error and is never performed** — and the exit path **discards the return value**, so the write is dropped **silently**.

□□□ **That is a write-drop primitive unprivileged guest userspace can aim at guest root.** A doorbell storm from one vCPU keeps the flag set; guest root's privileged writes on another vCPU disappear at random — the invalidate trigger (the guest then proceeds with stale translations), the interrupt acknowledgement (a storm, or a permanently masked engine), the message-queue kick (the request is never noticed, and enough of those mark the device for reset and **cost the guest its GPU until reboot**).

⇒ **The same regions must also set the per-region guard opt-out, and a region that runs lock-free with the guard still enabled must fail at startup.** □ The two changes are one change; shipping the first without the second is worse than shipping neither. ◻ Without it, every trapped write runs under the same lock as memory-region commits and the main loop, and the tail argument above is false rather than merely optimistic. ◻ And the obvious alternative does not work: the kernel's doorbell-style fast path **discards the data word**, and a doorbell carries a token we must read.

□ **We do not manage vCPU blocking — we stay out of it.** A vCPU blocks routinely and correctly when the *guest* halts and KVM parks it. What is forbidden is a block **we** impose: the guest did not ask, cannot see it, and its scheduler believes that core is running.

4. Two privilege boundaries

The outer one is the usual: the guest is untrusted. The inner one is the design's organising idea.

□□□ **Unprivileged guest userspace writes the doorbell directly.** The threat is not the guest corrupting itself from root — it is a sandboxed process in the guest reaching guest root.

Three facts make it unfixable by any check at the trap:

1. The doorbell page is **one memdesc per GPU**, DUP'd to every client, 64 KiB, doorbell at `+0x90` — and RM waives its own privilege check on it by name, `MEMDESC_FLAGS_SKIP_REGMEM_PRIV_CHECK`.

2. The token is a bare concatenation of runlist id and channel id. No capability.
3. The trap has **no identity to check**. The usermode window is a BAR0-relative register offset, and the driver's mmap gate admits it *because* it is inside the register BAR. Guest kernel and guest userspace reach one GPA, one memslot. CPL is readable; *which process* is not, and that is what an authorization decision would need.

□ **What a hostile ring actually buys is cycles.** The attacker supplies 32 bits and nothing else; everything we then act on comes from memory only the victim can write. Our cost profile is hardware's: ring anything, pay a redundant look.

⇒ **So the rule is about reach, not inspection:**

plane	writable by	therefore
doorbell	unprivileged guest userspace	fixed-size, idempotent, non-degradable , needs no identity
privileged registers	guest root only	an ordered ring that must not overflow — root harms only itself

⊗ **The test for any trapped page: can unprivileged guest userspace write it?** If yes, it gets no queue, no degradation mode, no structure shared with another guest process. That set must be **derived** from which objects RM maps to non-kernel clients, not hand-written — a page missing from the list defaults to the privileged arm, which is the bug.

5. The vCPU trap path

Only **writes** trap. Reads are served from ordinary DRAM the guest reads directly, with no exit and no code — because a vCPU inside an MMIO exit is not preemptible, and driver init polls some registers thousands of times.

□ **Not every read, though.** Roughly **524 of 4096** BAR0 pages must still read-trap — the framebuffer window resolves through a latch, and the firmware-boot pages are state-machine state. **The read-trap set is an allowlist, written down, exactly like the write list.**

⊗ **The timer page is not one of them.** On Turing and newer the kernel's clock is the pair in the *usermode* window — which is already the read-only memslot over live host time — and the legacy page at the old offset is touched only at boot and resume, never read at runtime. ⇒ It is a **computed shadow**: constants for the privilege mask and tick frequency, and the two time-register writes **refused by name**, so guest and host cannot drift onto different timebases. □ One residual is handled the same way as the doorbell page: if a guest client ever asks for its own CPU mapping of the timer, install a read-only memslot over ours, lazily, by handle.

`nvkvm_trap_write(bar, off, val):`

```

if (bar, off) == chip.doorbell:           # generated per die/arch; Hopper+ maps it over BAR1
    tok = val & chip.token_mask
    w   = &token[tok]
    r   = route_of(w)
    if r == PASSTHROUGH: write the host doorbell INLINE. No queue, no wake, no lock. Return.
    if r == UNKNOWN:     Return.           # no bit, no bump, no wake
    prev = fetch_or(w, RUNG)                # AcqRel
    if prev had RUNG:     Return.           # already pending
    set the rung bit; bump work_seq; maybe write(eventfd)
    Return.

elif chip.userspace_mappable(bar, off): Return.           # do NOTHING

else:                                     # privileged: guest root only
    if readable(off): shadow_write(off, val, write_semantics(off))
    append to the privileged ring
    bump work_seq; maybe write(eventfd)
    Return.
```

□ **The middle arm is the one the threat model needs.** The doorbell page is 64 KiB and the doorbell is four bytes of it; every other offset on it is guest-userspace-writable, and without this arm an unprivileged process pushes unbounded garbage onto the plane reserved for guest root.

□ **UNKNOWN must return doing nothing at all.** Bumping the sequence for an unowned token lets an unprivileged process keep every worker spinning: workers register as polling only if the sequence is unchanged, so a token nobody owns, rung in a loop, prevents them ever parking.

5.1 Two structures, because one cannot do both jobs

structure	shape	job
token word	one u64 per token: route, state, applied_seq stamp, host token	every CAS. No neighbour can fail it
rung bitmap	1 bit per token + a summary bit per 64	scanning only , never CAS'd

Sizing comes from what the register can **express**, not what is legal: the decoded fields are 12 bits of channel and 7 of runlist on Ampere, plus a doorbell-type bit on Blackwell — 19 to 21 bits. ⇒ Token words are 4-16 MiB and are **never scanned**; the bitmap is **64 KiB** with a **1 KiB** summary and is what a worker walks. The token is **masked**, not validated — masking is what the hardware does with undecoded bits, and it removes an error path.

□ **The bitmap is a hint; the word is the truth.** A bit set over an IDLE word costs one look. □ **Clear the summary bit first, then re-read the word.** Clearing after a scan loses a token that arrived during it.

5.2 The token's four states

```
vCPU:    stamp the ring position, THEN fetch_or(RUNG)          # order matters — see §5.6
worker:  CAS RUNG → BUSY      (fail ⇒ not ours)
        act
        CAS BUSY → IDLE      (fail ⇒ BUSY_RUNG → CAS BUSY_RUNG → BUSY, act again,
                             bounded: after K rounds, CAS BUSY_RUNG → RUNG,
                             re-publish, and move on)
        cannot act yet ⇒ CAS BUSY → RUNG, then re-publish
```

⊗ **“Re-publish” means the bitmap bit AND the summary bit, in that order.** Setting only the summary loses the token **permanently**: the next worker clears the summary, finds the group word zero, and moves on — and a further ring from the guest returns early because RUNG is already set, so it produces no bit and no wake. ⇒ **Publication is always bit-then-summary, and the trap and the put-back use the same order.**

⊗ **The re-act loop is bounded, because it is otherwise a livelock an unprivileged guest process can hold.** A process ringing its own channel in a tight loop keeps a worker in *act* → *CAS fails* → *act* forever; with enough channels it pins every worker and the guest kernel's own scrub and UVM channels starve — the inner boundary §4 exists to defend. ⇒ After K rounds the worker republishes and moves on. □ That is a timeslice, and it is what hardware does for the same reason.

⊗ **A token needs a retired state, or channel recycling is a use-after-free.** Without one: the guest frees a channel while its token is *BUSY*, the free path drops the twin, the guest's driver **recycles the channel id**, and the new channel's first ring lands on a word still carrying the old route — while the old worker is still reading what it believes is a pushbuffer. ⇒ Free transitions *IDLE|RUNG* → *DEAD* and **waits out busy** before dropping the twin; allocation transitions *DEAD* → *IDLE* with the new route.

□ **A bit that says work exists is not a bit that says you may touch the hardware.** Without exclusion, two workers walk one channel against one cursor: *[c0,p1]* executes twice, or the cursor moves backwards and the front-end reads it as a ring wrap. ⊗ This is owed to the **victim's own concurrency** — two legitimate rings race identically. An adversary only widens the window.

□ **“Found but unactionable” counts as no work** for the purpose of sleeping; otherwise a worker spins on a token whose enabling register write also needs a worker.

5.3 The wakeup word

```
worker_vcpu_poll : u64 = { work_seq : 56 (HIGH) , workers_polling : 8 (LOW) }
```

vCPU: publish, then *fetch_add*(1 << 8); if the pre-value had pollers, write the eventfd. Worker: read seen **before** scanning; scan; register as polling **only if the sequence is unchanged**; otherwise rescan. That is *prepare_to_wait*-then-schedule, and it removes the question of who clears a flag and when.

work_seq is the **high** half: a carry then falls off the top of the word instead of landing in the poller count as a phantom poller that makes every later trap pay a syscall. 56 bits is not about global wrap — it is about wrapping *inside one*

scan, and every vCPU CASes the same cacheline, so the aggregate rate is capped by ping-pong at $\sim 10^7$ /s. That is ~ 228 years.

Orderings are load-bearing and x86 TSO hides their absence: *AcqRel* on the poll word both sides, *Acquire* on *seen*, *Release/Acquire* on indices and claims. The rule: **every write to a work source happens-before the bump, and every seen load happens-before the scan.**

□ The eventfd is a **sum, not a queue of wakeups** — *EFD_SEMAPHORE* | *EFD_NONBLOCK*, and a wake must reach **one** waiter, not all of them. ∅ A plain write wakes every non-exclusive waiter, so at 255 workers a single doorbell wakes 255 threads and 254 of them find nothing.

∅ **And the drainer parks on its own word, not the workers'.** Sharing one word with a wake-exactly-one policy means a privileged register write can wake a **worker instead of the drainer** — and the register write then waits for an unrelated doorbell while the guest spins on a trigger. = **Two words, signalled by the two arms of the classifier**, which are disjoint by construction. □ This does not reopen the multi-GPU objection in §9.1: that is about workers registering on more than one word, and they still register on exactly one.

□ **No handler may wait on another queued item**, or a collapsed wakeup becomes a hang rather than latency.

5.4 The privileged ring

One bounded **lock-free MPSC ring**, fixed capacity, preallocated and **prefaulted** — growth means the allocator on a vCPU. The claim is **conditional**: read the cursor, test fullness, take the slot by CAS; on full, poison and return **having claimed nothing**. ∅ A *fetch_add* claim that then waits for the consumer is the vCPU waiting on a worker in lock-free costume; a claim-then-bail strands the consumer at that slot forever.

One **dedicated drainer**, spin-then-park, peek → apply → commit, so a failed apply retries at the head and *applied_seq* is a clean monotonic number other paths fence against.

Why the order matters at all: a guest thread on one vCPU writes *PDB_LO/PDB_HI* under an RM lock and releases it; another thread on another vCPU takes the lock and writes *TRIGGER*. On hardware the first trap returned before the lock released, and PCIe writes to one function are ordered. Per-vCPU rings preserve only per-vCPU order and fire the trigger against a stale base.

∅ **Full ⇒ poison the device**, never wait: a sticky error state, further writes dropped, a guest-visible fault. Dropping one write silently leaves state a later, differently-privileged guest process inherits.

Capacity: 4096 entries per GPU, 64 KiB, prefaulted. The bound is derived, not guessed: every runtime write funnels through one RM entry point, and RM **serialises its own register sequences with its own locks**, so in-flight count does not scale with vCPUs. A lock cycle is ~ 7 writes (~ 13 on Turing), an interrupt clear is one, a TLB invalidate is 2–4 **and the guest pre-polls before writing**, and the RPC kick allows **at most one outstanding**. Realistic maximum: **under 64**. = 4096 is a $60\times$ margin; sustained occupancy above 25 % is a defect to investigate, and the high-water mark prints at teardown.

∅∅ **One class must never enter the ring, and it would overflow any ring that exists.** On Turing and GA100 the firmware images are loaded through **auto-incrementing falcon data ports** (*IMEMD/DMEMD*, and Hopper's *EMEMD*), unpolled, ~ 256 writes per kilobyte ⇒ **16 000–65 000 back-to-back writes** during boot. = They are classified *data_port* and applied **synchronously on the vCPU** as a store into the falcon image shadow at the address the companion latch names. That is a memory store, it is in the privileged arm, and without it the design cannot boot on those parts.

5.5 The shadow

A register the guest can read back is written to the shadow **synchronously, in the trap**, before anything is queued — because reads come from that page and a deferred write leaves the guest reading stale state. The sharp case is interrupt masking: the ISR writes the mask then reads pending, and a deferred mask delivers an interrupt the guest already masked.

□ **A plain store is wrong for four of the five kinds**, so the register descriptor carries *write_semantics*:

kind	behaviour
plain	store
wlc	<i>fetch_and(!bits)</i> — a load-store loses a worker's concurrent set
<i>write_only_port</i>	the readable effect is on a different register
trigger	the guest writes 1 and spin-reads until 0 ⇒ □ cleared after the work is done , by whoever did the work — see below

kind	behaviour
data_port	an auto-incrementing image port = a synchronous store into a shadow image (§5.4)

⊗ **write_semantics cannot be generated from the published access codes, and that is measured.** The interrupt-pending register (write-1-to-clear) and its enable-set port carry the **same** code; the invalidate trigger reads as plain read-write. ⇒ Read/write-only-ness is generated; the rest is a **hand-maintained per-family overlay of about a dozen names** — the interrupt leaf and its set/clear ports, the invalidate triggers, the falcon interrupt-clear and transfer-command registers, and the data ports. □ The overlay is small, but it is hand-maintained, and it must fail the build when a register in the generated set has no entry.

□ That last row is why the drainer *does* write some guest-writable shadows: it owns their completion. It must not write one the guest also drives.

⊗⊗⊗ **But the drainer must never WAIT for that completion, and a naive reading of the rule makes it wait for the slowest thing in the system.** An invalidate trigger completes only when the VA manager has applied the diff — which waits on the GPU walker and on host mapping calls taken under the host driver’s own lock. ⇒ If the drainer blocks there, **everything queued behind it stalls**: the message-queue kick, the interrupt re-arm, every later register write. A co-tenant holding the host GPU lock for a few seconds then lands squarely in the next hazard.

⇒ **The drainer hands the trigger to the thread that does the work and moves on.** It keeps two counters — what it has *issued* and what has *completed* — and the owning thread clears the shadow when its work is done. Doorbell workers fence on *completed*, and only for the polled class.

⊗⊗⊗ **And the clear must name what it completes, or a slow invalidate silently corrupts.** The guest’s invalidate spin has a timeout of a few seconds, after which **the driver proceeds anyway, with stale translations and no error** — that is its documented behaviour, not a bug we can fix. So: the guest times out on invalidate *A* and continues; it later issues invalidate *B*; our work for *A* finishes and clears the trigger; the guest reads zero and concludes *B* is done. **It is not.** ⇒ The shadow carries the ring position of the write it belongs to, and the clear is a compare-and-set against that position — never a blind store. □ **And add the tripwire:** a trigger outstanding past a fraction of the guest’s own timeout budget **faults the device loudly**. Letting the guest soldier on with stale translations is the failure mode that produces wrong numbers rather than a crash.

⊗⊗⊗ **A late unmap is a SECURITY event, not a latency problem, and an attacker can cause one.** The guest’s invalidate wait is bounded; ours is not. Landing an unmap means a host call taken under the host driver’s own device lock — and **a hostile channel can hold that lock**, by faulting the host GPU in a loop and making the driver run recovery. ⇒ The guest times out, scrubs the page, hands it to another process — **while the attacker’s host mapping still resolves to it.** ⇒ □ **If any unmap in a refresh lands later than the guest’s own timeout for that driver version, tear down every host channel that could still hold the stale translation before applying anything else**, and count it. A non-zero count is red in the suite, not a warning. □ And issue unmaps as soon as the walker reports a cleared leaf, rather than only at the invalidate.

5.6 Doorbells versus registers, across the boundary

Order holds *within* the ring. Doorbells go around it — passthrough hits hardware inline, managed sets a bit another thread consumes. ⇒ A doorbell can be serviced against pre-write device state.

The fence: the doorbell trap stamps the current ring position into the token word, and the worker services it only once the applied position has caught up. The **worker** waits; the vCPU never does. ⇒ The doorbell invariant holds: *queued now* ⇒ *eventually scheduled*, *never blocks*.

Three details are load-bearing:

- ⊗ **Stamp before setting RUNG, or fold both into one atomic.** They are two read-modify-writes; if RUNG lands first, a worker can claim the token, read the *old* stamp, believe the fence is satisfied, and act against pre-write device state — the exact thing the fence exists to prevent.
- ⊗ **Compare in modular order, not with a maximum.** The stamp shares a word with route, state and the host token, so it is narrower than the ring counter. A plain `fetch_max` never wraps back, so after the stamp space rolls over the token becomes **permanently unserviceable** — a hang that appears after weeks of uptime and nothing else explains. ⇒ Compare as a signed difference and store-if-ahead in modular order.
- ⊗⊗ **The drainer must wake the workers after it commits.** Otherwise: a worker claims a token, finds the fence unsatisfied, puts it back and parks; the drainer applies the write and advances; **nothing rescans**, because parking is only ever ended by a new bump. ⇒ A guest that rings once and waits on a semaphore — any single stream synchronise — hangs. The drainer bumps the sequence and signals after every batch that advances past a stamped position.

□□□ **Passthrough needs no fence, and the obligation is discharged by enumeration.** Every privileged runtime write on a GSP client falls into one of four classes, and none of the first three can change how hardware services a channel at the moment of a ring:

class	why it cannot matter
polled	the guest spins on a read-back — invalidate triggers, L2 invalidates, falcon transfer and control. □ The drainer clears the polled bit only after the host-side effect is complete , so the guest cannot ring until it observes the clear. Self-fencing.
rpc-fenced	the message kick allows one outstanding and the guest waits for a reply we send after acting. ⇒ Channel alloc, bind, schedule and preempt all ride here — ∅ there is no register path for runlist submit or channel enable on a GSP client
irrelevant	interrupt tree, timers, fault-buffer cursors, mailboxes, data ports
refused	the fault/recovery registers — channel-RAM and runlist preempt. ∅ Unimplementable from host userspace anyway , and already excluded by <i>delivery, never recovery</i>

⇒ **[PROPOSE] Make it a build-time property, not a proof in prose:** every privileged register in the generated descriptor set carries a `fence_class`, and **an unclassified register fails the build**. A new die then cannot reopen the hole silently. □ Keep the managed-channel stamp as a free tripwire and assert the wait count is **zero** at teardown — a nonzero count means the enumeration is wrong.

5.7 The doorbell page itself

Mapped as a **read-only KVM memslot over the real host doorbell page**: reads are hardware with no exit, writes trap into the path above, and when we ring we write the host page directly.

On Hopper and newer, unprivileged clients map the doorbell over **BAR1** instead — `bBar1Mapping` is not privileged, and both UVM and NVIDIA's own pushbuffer library request it. ⇒ **BAR1 is never trapped, with exactly one exception: the page the guest maps the usermode object at.** We identify it by **RM object handle** — we serve the allocation, so we hold the handle and the flag — and use the page-table diff only to confirm. Identifying it by “it looks like the VF register block” is a pattern match on guest-chosen data.

□ On Hopper+ the guest kernel still rings BAR0 while userspace rings BAR1 — **two pages, two GPAs, two privilege domains**. Where the hardware gives us that separation, use it.

6. Memory and addresses

Guest video memory is one host allocation, sized by a command-line parameter like guest RAM. The guest framebuffer offset **is** the offset into it. Guest system memory is the hypervisor's memfd through a static layout.

Mirror, never adopt. We reproduce the guest's page tables in our own; we do not take over its. Three independent reasons: adopting races the host driver's own tables and invalidates; the host driver exposes no userspace interface to participate in its locking, so adopting needs a host kernel module and a deployment bottleneck; and a translation is needed anyway, because guest physical is not host physical. ⇒ Mirroring is also **rootless**, which is the posture the product needs.

The diff. A GPU-side walk of the guest's page tables produces a per-VA-space delta — which spaces changed and which entries to apply. Previous state lives in video memory, held by the walker itself. The VA manager executes the delta as `mmap/munmap`, batched with the TLB-defer flag on every call but the last. □ The defer batches the barrier; it does not remove it — a refresh still happens last, **and it is issued unconditionally, including on a failed batch.**

∅∅∅ **There is more than one publish trigger, and the one the design named is not the one CUDA uses.** The register-based invalidate is how the guest *kernel* publishes. But the unified-memory path writes its page tables **with the copy engine** and issues its invalidate **as a pushbuffer method** — it never touches that register. ⇒ A design triggered only by the register **never mirrors a managed-memory unmap**, and the freed guest page is handed to another process while the first process's host mapping still translates to it. **That is the cross-process leak, reached by ordinary CUDA, with no race required.**

⇒ □ **The decoded invalidate method on a translated kernel channel is a first-class publish trigger**, carrying the same barrier as the register path: the pushbuffer position is held until the unmaps have landed.

○○ **And the method must be stripped from the forwarded stream, not passed through.** It is a **privileged** method: a channel that is not privileged **faults** when it executes one, and our host twins are unprivileged by construction — the host driver grants that privilege only to kernel or administrative clients. ⇒ Forwarding it would fault our own twin. We consume it as a trigger and do not submit it.

○○ **The walk must be ordered against the guest’s own page-table writes, and one case is not naturally ordered.** The guest’s memory manager writes video-memory page-table entries **with the copy engine, inside a pushbuffer**, and issues its invalidate **in the same pushbuffer on the same channel**. ⇒ If we trigger the walk when we *decode* the invalidate method, we read the tables **before the engine has written them**. The split is: submit the pushbuffer prefix up to the invalidate, **wait for its completion on our twin**, walk, then continue. □ This is an ordering the design assumes and must establish.

○○ **And the diff is two-phase, or a partial failure desyncs the mirror permanently.** If the walker commits its new state when it walks, and the VA manager then fails part-way through — host address space exhausted, an allocation refused — the unapplied entries are **never retried**, and the mirror believes they are done. ⇒ The walker commits **only what the VA manager acknowledges**.

A VA space is the object; the page-directory base is an attribute of it, per GPU. RM’s identity is the object and a monotonic unique id; the base is per-GPU, mutable (relocation is explicitly repeatable), can be **absent** — hardware encodes that case — and is not unique. ⇒ Identity is the object; carry the base as a mutable, possibly-absent, per-GPU attribute used to seed a walk.

We write the roots, because the page-directory update callback is a no-op inside a guest on GSP parts — instance blocks are written by the firmware, which is us. Where each root comes from:

space	root arrives as	aperture	walked by
user VA space	a control carrying the reserved-PDE table , whose level-0 entry is the top-level directory. Sent for every space, because the split-VA-space default is on	declared in the same message	the GPU walker (vidmem) / CPU (sysmem)
UVM’s external root	a control carrying {physical address, entry count, aperture, VA space, channel}. □ It arrives as a control, not as the dedicated message — that message is stubbed out on GSP clients	in the message	same
BAR1	∅ ours . We allocate it and declare the address; the guest adopts it and writes entries directly. No message exists	□ <i>framebuffer to the guest; host RAM underneath</i> — see below	CPU, at RAM speed
BAR2	∅ ours . We declare the address; the guest sends back its encoded entry value for slot 0	same	CPU, at RAM speed
instance block	∅ not a root . The guest allocates it and tells us where; we write the base and aperture into it	same	never walked

6.1 □□□ The guest’s BAR tables are metadata. Nothing but us ever reads them.

□ A CPU read of **real** video memory runs at roughly **48 MiB/s** — flat, regardless of what you do. ⇒ “*We walk the BAR tables with the CPU*” would be a serious mistake **if those tables lived in video memory**, and under the single store the guest’s framebuffer is one real device-local allocation. So the question is which side of that line they fall on.

□□□ **They fall on ours, and the reason is that nobody else reads them:**

- On real hardware the **GPU's own MMU** walks a BAR2 table when the CPU touches BAR2. In this design **there is no such hardware access** — the guest's BAR2 window is an aperture *we serve*, and its BAR1 access is a **KVM memslot we install**. No engine ever fetches these entries.
- The **host GPU** has its own BAR1 and BAR2, owned by the host driver, with their own tables. The guest's are a different object entirely.
- ⇒ The guest's BAR page tables exist to tell **us** what the guest wants mapped. They are **metadata, consumed once, by software**.

⇒ **So place them where reading is cheap.** The guest must *believe* they sit at framebuffer offsets — we declare those addresses, and it writes through apertures we emulate, so **we choose where the bytes actually land**. Back them with **host RAM**. The CPU walk is then a RAM walk, and the 48 MiB/s figure never enters the picture.

□ **The window itself is a memory slot, and slot churn is the part that bites at scale.** Deleting and re-adding a slot per window change is expensive on the host, and it interacts badly with other devices that count slots. ⇒ **Reserve a fixed host address range per window and remap into it in place — never delete the slot.** The kernel invalidates just the changed range, the slot count is constant, and nothing else on the machine has to care how often the guest re-points a window.

□ **This is also why the GPU-side walker is not the answer here**, even though it is the right answer for user VA spaces. Those tables are **guest-allocated in real video memory** — we do not choose their placement, and a CPU walk of them is the 48 MiB/s path the walker exists to avoid. ⇒ **Two page-table populations, two mechanisms, and the discriminator is who chose the placement:**

tables	placed by	live in	walked by
user VA spaces, and everything the guest allocates	∅ the guest	real video memory	□ the GPU-side walker
BAR1, BAR2, instance blocks	□ us	host RAM	the CPU, at RAM speed

□ **The trap this closes:** “*it is at a framebuffer offset*” and “*it is in video memory*” are not the same statement in this design. The first is what the guest is told; the second is a placement decision we make. Conflating them is what makes a CPU walk look unavoidable when it is not.

□ **Aperture decides the verb, but it is not the whole leaf.** A page-table entry carries kind, read-only, atomic-disable, privilege, volatility and a peer id — and the host mapping must carry them too. ⇒ The walker's leaf record is {aperture, address, kind, ro, atomic, priv, vol, peer}, and the VA manager maps flags from it. ∅ A leaf record that is only an address will produce mappings that are wrong in ways that fault late.

□ **The verbs**, once the aperture is known: **vidmem** is a fixed-offset DMA map of a slice of the single store — the backing already exists, so nothing is allocated; **sysmem** is an OS-descriptor object over *our* host pointer into the guest-RAM memfd, plus the same fixed-offset map. ∅ The guest's own pointer never appears in a host call, which is why the memory class that would carry one stays refused by name. **Peer** is a third verb and belongs with multi-GPU.

∅∅∅ **The system-memory resolution must admit ONLY true guest RAM, and this is a security boundary rather than a correctness one.** Our own structures — the register read shadow, the doorbell bitmap, the boot pages — are host memory installed as guest-physical memslots. ⇒ A guest page-table entry naming one of *those* addresses, resolved by a layout that maps guest-physical to host pointers generically, would **pin our own state and hand it to the GPU as a DMA target**. The guest could then have the engine write the bits the drainer owns. ⇒ □ **The layout resolves guest RAM blocks and nothing else. Any other guest-physical address is a refused leaf, by name.** □ Same shape as refusing the memory class that carries a caller pointer: *the guest may name its own memory, never ours*.

∅ **And that class refusal is decorative — this is the real guard.** The class never reaches us; it is handled inside the guest. ⇒ **The only path by which a guest-chosen address reaches a host call is this leaf**, so it is the one that must carry the bound. The design states a bound for video-memory leaves; **the system-memory leaf needs the same bound stated, not implied**.

∅ **The leaf's other bits are an allowlist, not a translation.** Forwarding a guest-chosen page **kind** would mint host mappings from a guest value and touch device-global compression state — a guest-to-host coupling in the same family as forwarding a flag word. ⇒ We translate **aperture, address, read-only and page size**; every other bit is **refused by name and counted**, and the kind is fixed to the store's.

6.1 BAR1 and BAR2 are split between us and the guest

□□□ **The guest owns PDE3[0]; we own PDE3[1]; and our table is the one hardware walks.** The guest rewrites its own page-directory cache to **our** address, which it learns from our static-info reply. So:

- We allocate **both** root pages in reserved framebuffer and **declare their addresses** in `GET_GSP_STATIC_INFO`.
- The guest's TLB invalidates then name **our** base; our BAR2 walker roots at our page with entry 0 set from the value the guest hands us.
- ☐ **BAR1 has no update message at all** — the guest adopts our root page and writes entries into it directly.
- ☐ **Nobody sparsifies BAR1 unless we do**. An unpopulated entry there is sparse, not an error.

⇒ The framebuffer layout — root pages, the protected firmware region, reserved rows — is **ours to declare, before static info can be answered**.

7. Channels

A channel is a pushbuffer, a GPFIFO ring of 8-byte entries, and a small block holding the ring cursors. The guest appends an entry, bumps its producer cursor, and rings. ☐ The consumer cursor lives in memory mapped to the owning process and nobody else — which is why hardware can treat a hostile ring as harmless, and why we must too.

kind	contract	what happens
Passthrough	ring and return; may run on the vCPU	the guest's ring is in real GPU memory; hardware fetches it. ☐ We never read a byte of it
Translated	schedule and return	every operand rewritten into our VA space, submitted on our host channel. The GPU moves the bytes ; the completion is translated in address only
Emulated	schedule and return	we implement the function

Birth is at allocation, never lazy — RM zeroes a caller-supplied cursor block at allocation, so adopting at first doorbell wipes the cursor that just rang.

Kernel channels are translated, not emulated. The scrubber and UVM's own channels do real work on real memory; running them on our CPU is how a guest process reads another's freed pages.

☐☐☐ **An untranslatable operand must never become a deliberate fault on a kernel channel**. The unified-memory driver treats **any** channel error as **globally fatal** — one fault kills CUDA for **every process in the guest** until the driver reloads. ⇒ A design that forwards a translation miss as a sentinel fault hands unprivileged guest userspace a way to kill the whole guest's GPU stack. **On a kernel channel we refuse the submission and poison the device** — root-visible and honest — rather than faulting it. ☐ And any guest memory region we cannot back must be **refused at startup**, not discovered as a translation miss at run time.

☐ **Sizing and decoding are different jobs**. We *size* every pushbuffer method form so the stream never desynchronises, and *decode* only what we model. An undefined form decodes to nothing rather than to a guess. And a method is not a unit of meaning — a copy is five method runs and the launch carries no operands, so only a stateful walk over a run produces a fact.

8. Completions and interrupts

kind	completion
Passthrough	nothing. We do not inspect the channel and do not care when it finishes
Translated	nothing. The semaphore address was forwarded; the GPU writes it
Emulated	forged at the end of our own call — there was no GPU work

☐ **"Forge" is licensed only where there was no work**. A completion written for work that did not happen is how a scrub becomes a leak.

☐☐ **What we may send the guest is an allowlist, enforced at build time**. We know one message crashes the host operating system on a Windows guest — but it is not the only one that hurts: at least three other outbound

messages and paths cause the guest’s driver to **mark its device for reset**, which costs the guest its GPU until it reloads. One of those paths is simply **letting a reply take too long**, repeatedly. ⇒ **Three messages are permitted — boot-complete, the generic completion carrier, and channel-teardown — and the build fails on any other.** □ And every synchronous reply carries an internal deadline **well under** the guest’s own patience, because the timeout path is reachable by anything that starves the thread drawing replies.

Interrupt registration is per engine, not per channel and not per semaphore: the event object requires a subdevice notifier, the index names an engine, and the channel class has no completion event at all. ⇒ An interrupt is a **broadcast wake to every waiter on that engine**, carrying no identity.

□ **The waiter owns the race**, three independent ways in RM’s own code: its semaphore-surface registration re-reads under a lock and returns *already-signalled*; callers loop on that; and the wait path never trusts the event, looping on the notifier word and using the event only to decide when to re-check. ⇒ We fire on **genuinely new completed work** and need not fire retroactively. □ The converse binds: once armed, a later release **must** produce an interrupt. There is no level-triggered fallback.

□ The channels we care about most **poll** — UVM spins, and the scrubber’s blocking waits loop on the semaphore word.

8.1 ⇒ We forward the host’s edge, and never decode the request method

⊗ **We do not decode the pushbuffer method by which the guest requests an interrupt.** Three facts make it unnecessary:

1. **The guest learns its vector from us, not from hardware.** Non-stall vectors are reported only through a control we serve; the device-info table carries no vector at all.
2. **Servicing is one register write** — a write-1-to-clear of the pending bit — followed by a **per-engine broadcast** to every registered waiter. No per-channel work, no message. □ Enables are set once at state load and never masked at runtime, so the pending bit is a sticky latch behind a permanent enable.
3. **The method executes on the host GPU** for passthrough and translated channels, and the **host driver broadcasts the same edge** to every registered host waiter.

⇒ **Mechanism:** our host client registers a per-engine non-stall event on the host twin for each copy engine, graphics and host, and those descriptors join the drainer’s poll set. On wake we set the pending bit for the vector **we declared**, and raise the guest interrupt if it is enabled. Emulated channels set the same bit at the end of their forged completion.

□ **Why this is correct rather than merely convenient:** the guest’s contract is “*the engine advanced = broadcast; the waiter re-checks memory.*” Forwarding the host’s edge yields a **superset** of the required edges — other host clients’ work on that engine also wakes us, which is redundant re-checking, and is exactly what hardware does across processes. Missing edges are impossible while the host driver notifies us at all.

□ **One invariant closes the only known hole:** when the guest re-arms the top-level enable, the drainer must **re-evaluate pending and re-raise** if an enabled bit is still set. The host’s non-stall path is strictly one-shot, so nothing else recovers a dropped edge.

⇒ **This is deferrable past the first compute**, because the things that would need it poll. □ It is deferrable **only if the absence is red rather than silent:** the suite gets an arm that uses a blocking-sync wait, so the gap fails a test instead of hanging a workload.

9. The host side

Host RM objects are owned in-process now that isolates are gone. **We author every host call;** our host-verb signatures do not accept a guest flag word, so a guest-chosen bit cannot reach RM’s interpretation of it.

□ **Host RM recovers channels whether we want it or not.** A fault on our twin gets the channel torn down by the host driver, and the guest must be told through the mechanism it already understands, then allowed to run its own recovery and re-adopt.

□□□ **Lifetime: two host descriptors, two lifetimes, and the kernel does the freeing.**

descriptor	lives for	owns
A	the VM	the single store, the host register mappings, the walker’s GPU context

B	one guest driver instance	every twin — devices, VA spaces, channels, engine objects, events, mappings
---	----------------------------------	---

◊◊◊ **But `close()` is not a synchronous free, and relying on it leaves exactly the hazard it was supposed to close.** Two mechanisms defeat it:

- **release runs on the last file reference**, not on `close`. Every in-flight call on that descriptor and every mapping made through it holds one. $\Rightarrow$ `close` returns while teardown has not started.
- ◊ **The driver's close path defers the whole cleanup to a kernel thread** when its interruptible wait fails — which **a pending signal causes**. VMM threads carry signals routinely. $\Rightarrow$ The deferred path is not an edge case; it is the normal case under load.

$\Rightarrow$ ◻ **So we free explicitly and then close.** Unmap every mapping made through the descriptor, join every in-flight call, **block signals**, issue the **explicit free of the client tree** — which is synchronous and ordered by the host driver's own dependency rules, preempting and unbinding channels — and only then close the descriptor.

◻ **Why this matters rather than being hygiene:** a still-scheduled host channel whose ring lives in guest RAM the guest has since reused is **a live DMA engine writing into another guest's memory**. The guarantee has to be *"the engine is stopped before the memory is reused"*, and only the explicit free gives that ordering.

Triggers: the guest's unload message, a device reset (`system_reset`, function-level reset, `reboot -f`), and re-entry of driver init with the protected region still up. Our own order is: quiesce workers, reset the ring, drain the VA manager, drop twin references, **then close**. ◊ On VMM exit or crash the kernel closes both descriptors and the host driver frees everything — **a leak across runs is not possible without a host driver bug**. ◻ That is true of *leaks*; it is not a substitute for the ordered teardown above, because a crash gives us no chance to run it.

◊◊ **And teardown must reset the walker's own state.** Its previous-state snapshot lives across a driver instance; if it survives while every mapping is dropped, the next instance's first diff reports *"unchanged"* and maps **nothing** — a second boot that faults for reasons the first did not. $\Rightarrow$ Resetting it is part of the teardown order, and the gate is the design's own rule: **run every gate twice in one process**.

9.2 ◻ Per-VM caps, because a shared host GPU is a shared resource

The host driver enforces no per-client quota. $\Rightarrow$ On a host with more than one VM, one guest can starve the others by allocating twins — channels, engine objects, address spaces, and one host object per non-coalescable system-memory leaf. ◻ **We already told the guest how many channels it may have**, so that number is the cap, and the same applies to every other twin class: **refuse past it, by name, counted**.

◻ **And the guest's framebuffer aperture is a device-global resource on the host.** The window through which a CPU sees GPU memory is a fixed size shared by every process on that GPU. $\Rightarrow$ Size the guest's at startup **from what is actually free**, and make an unbackable page a **named refusal** rather than a silent hole — the guest reading an unbacked hole is a wrong answer, not an error.

9.4 ◻ Two host-platform prerequisites, stated because they are not ours to fix

- ◊ **Fine-grained runtime power management must be off for a GPU we drive.** The host driver **revokes every user mapping** when it powers the device down. Afterwards our inline doorbell write faults into a handler that stalls until a scheduled wakeup completes — a block **we** impose on a vCPU — and a read-only memslot over the same page makes the guest's exit return an error the VMM treats as fatal. $\Rightarrow$ A host prerequisite, plus a revocation flag the trap checks before touching any host page.
- ◻ **Ballooning and memory hot-unplug must be disabled** for the guest, because we pin guest pages for the GPU. Discarding a pinned page is silent guest data loss.

9.1 Multi-GPU: what is per device and what is per process

per GPU	per VMM
the register shadow and classifier, keyed by (<code>gpu</code> , <code>bar</code> , <code>offset</code>); token words and rung bitmap (token spaces overlap across GPUs); the ring and its drainer — ordering is per PCI function; the boot state machine; the interrupt tree; the host twin and its events; the single store; the walker's GPU context; both BAR root pages	the worker pool ; ◻ one wakeup word and one eventfd — a worker registers on exactly one word, and N words would need N atomics and reopen the lost-wakeup proof; one VA manager, which also gives cross-GPU ordering for peer mappings for free; the object graph, since client handles are one namespace

∅ **Refuse broadcast device groups by name.** The “one VA space, many bases” case arises only there, and CUDA never uses it. □ **Peer access is a third leaf verb:** an entry whose aperture is *peer* and which carries a peer id maps as a peer association between our twins plus a fixed-offset map of the other store’s slice. The guest’s peer-id space is **ours to define**, because we serve the object that creates it.

9.2 Platform decisions

- ∅ **A guest IOMMU is detected and refused at device realize.** Every guest-supplied system-memory address we consume — message rings, page-table entries, cursors, semaphores, and the walker’s own reads — resolves through a guest-physical-keyed layout, i.e. **bypasses a guest IOMMU by construction**. Supporting it means a translation layer plus unmap tracking on every consumer, including a GPU kernel that cannot call the VMM. □ No CUDA or LLM guest needs one, and the defaults do not enable one. □ The product case that would reopen this is **Windows DMA protection**, and it is an OS-axis question, not a memory-model one.
 - □ **The GPU-side page-table walker runs in-process**, initialised **only from the VA manager thread** — the VMM blocks signals around thread creation, and the vCPU signal must not be inherited by a library’s own threads. It initialises at or after device realize, because the VMM daemonises before that. □ Under a sandbox profile that denies process spawning, the device nodes must already exist; that is a host prerequisite either way, and it is the one part of this not yet measured. ∅ The fallback if it ever fails is a CPU walk at ~48 MiB/s — **a degraded mode, refused by name for any parity claim**, not a supported path.
-

10. What is deleted, and why

Isolates and the whole IPC plane · the sandbox plane · per-process containers · the address table · joins · VA→phys translation on the submission path · publication epochs and the dirty gate · the CPU copy executor · the completion watch · the framebuffer probe · every on/off flag for something that no longer has two sides.

∅ **These are not edits inside the existing files** — they are the removal of the ideas those files are built around. Deleting them incrementally means running two architectures in one file set, which is how contradictory defaults got there in the first place.

□ **Keep measurements and gates; audit prose.** A comment asserting a default will rot. A comment asserting a dated measurement will not.

Part 2 — The plan

STATUS: LIVE, 2026-09-20 (w821). PROPOSAL — this is the thing to argue with.

∅ **This document is about the target, not the current tree.** [owner] “you don’t have to be extensive about what the rotten code now does — I’ll look at your intended target instead and argue with that.” THE_MACHINE.md stays in the bundle as the record; it is not the argument.

0. The three rules that shape everything below

1. □ **Rewrite in a fresh crate set**, old tree read-only. The deletions in THE_ARCHITECTURE_v3.md §8 are not edits — they remove the organising ideas the current files are built around.
 2. ∅ **A rewrite that cannot be graded cannot be landed.** Every phase names the arms of the 30-arm suite it must move, and no phase is “done” on a demo.
 3. □□□ **GSP plane only.** The no-GSP plane (THE_WINDOWS_AXIS.md §10) is **not in this plan**. It is additive reach, it is bounded at Ampere, and it must not be allowed to set any shape here.
-

1. The crate model — §10.4 discharged

crate	job	hand-written / generated	budget
kf-abi	NVIDIA constants: classes, controls, RPC ids, structs	□ generated from ogkm headers	~14 k (<i>gen</i>)
kf-chip	per-die descriptors: register offsets, token masks, doorbell BAR, format family	□ generated + a small family table	~3 k (<i>gen</i>)
kf-trap	□ the vCPU path . Shadow, three-way classifier, token table, wakeup word, register ring	hand	~2 k
kf-gsp	boot FSM, msgq geometry, element layout, RPC framing	hand	~5 k
kf-rm	object graph, alloc/control dispatch, the served tables	hand	~9 k
kf-mem	the single store, VA manager, page-table mirror, BAR windows	hand	~7 k
kf-chan	channels, doorbell service, pushbuffer decode, completions	hand	~6 k
kf-host	host RM verbs — ∅ authored, never forwarded (\$author_host_flags)	hand	~4 k
kf-core	VMM-agnostic device interface	hand	~1 k
kf-qemu	the QEMU shim; the only crate that knows about QEMU	hand	~2 k

⇒ ~33 k hand-written, ~17 k generated. □ The 50 k target is reachable **only because kf-abi and kf-chip are generated** — today’s kayfabe-abi is 43 588 lines hand-maintained, and that single change is most of the reduction.

□□□ kf-trap is ~2 k lines and is the whole security boundary. It should be readable in one sitting, have no dependencies but kf-chip, and be the most heavily tested crate in the tree. ∅ If it grows past ~3 k, something has leaked into it that belongs behind the trap (§48).

2. The phases

000 P-1 — THE QEMU BQL BYPASS. Item zero, and without it nothing below is measurable.

[NEW w821, from an adversarial review, verified against QEMU 9.2.0 in this workspace.]

THE_ARCHITECTURE_v3.md §2 claims the vCPU takes **zero locks**, and THE_CONSTRAINTS.md §48 builds a latency argument on it. 000 **Both are unreachable as the design stands, because QEMU takes the Big QEMU Lock for us, before a single line of our code runs:**

```
bool prepare_mmio_access(MemoryRegion *mr)          // system/physmem.c
{
    if (!bql_locked()) { bql_lock(); release_lock = true; }
```

Unconditional, on **every** MMIO dispatch. And the opt-out is gone: MemoryRegionOps::global_locking has **zero hits** in include/exec/memory.h at 9.2.0 — it was removed upstream.

⇒ **Every trapped BAR0 write runs under the same lock as memory_region_transaction_commit** (which is what the VA manager's BAR1/BAR2 memslot rebuilds do), the main loop, and every other vCPU's exit. §2.3's entire argument against a shared lock — lock-holder preemption, millisecond tails, the SCHED_FIFO deadlock — **applies verbatim to a bigger lock, held longer.**

0 R7, §41 and §48 are violated by the VMM before kayfabe starts. [owner, earlier] “vCPU threads must arrive without BQL” was the right requirement; it simply cannot be satisfied by configuration.

Build: a pre-BQL dispatch for our GPAs at the KVM_EXIT_MMIO site (accel/kvm/kvm-all.c), routing our BAR0/BAR1 doorbell pages straight into nvkvm_trap_write without address_space_rw. **Gate:** a microbenchmark showing a trapped doorbell write with bql_locked() == false in the handler, and worst_trap measured **without** a concurrent memory_region_transaction_commit inflating it.

0 **ioeventfd is not the escape**, and §2.5 excluded it for the wrong reason. The real reason is not cost — it is that **ioeventfd discards the data word** (or matches one fixed value). A doorbell carries a 32-bit token we must read. ⇒ Unusable, permanently, for this surface.

□ **Sequencing consequence:** every latency number taken before this patch lands is BQL-shaped and must be retaken. ⇒ **P-1 precedes P0.**

P0 — The harness, before any product code

Build: fresh workspace; the 30-arm suite running against it; the bare-metal baseline reproduced; the forwarded= per-token hardware gate wired; the GPU-free gate self-test. **Gate:** 0 the suite **runs and reports**, with **0/30 passing**. That is the correct result. **Why first:** every later gate is expressed in this harness. □ And the bare-metal baseline is what makes a guest failure *indict kayfabe* rather than the client.

P1 — kf-trap: the vCPU path

Build: the BAR0 read shadow (backable-run sweep, computed fill); the **three-way** classifier (doorbell / userspace-reachable-not-doorbell / privileged); the per-token 2-bit table + summary; worker_vcpu_poll; the lock-free MPSC register ring + **one dedicated drainer**; the synchronous shadow write for read-registers; the VA-manager thread. **Also in P1, because everything downstream needs them:** - □ **irqfd**. Workers cannot raise an interrupt through the QEMU path — it is BQL_LOCK_GUARD + msix_notify, so they would contend with the VA manager's memslot commits. ⇒ **irqfd, and it gates RmInitAdapter.** - □ **The swref descriptor generator**, with the write_semantics field (§2.3.1 rule 4). 0 **Its input is not parseable as C:** NV_CTRL_VF_DOORBELL_VECTOR 11:0 and NV_VIRTUAL_FUNCTION 0x0003FFFF:0x00030000 are not C expressions, so libclang yields nothing. It needs a **token-level hi:to + access-code parser**. That generator does not exist today. - 0 **The read-trap allowlist.** “Only writes trap” is **false for 524 of 4096 BAR0 pages** — PRAMIN, the GSP falcon pages, WPR2, NV_PTIMER. v3 has no analogue of §41 for reads. **Write one.**

Gate: □ **GPU-free.** Every interleaving in §2.4 as a test: lost-wakeup, summary-clear race, double-take, BUSY-RUNG, ring-full poison, cross-vCPU register order, and the unowned-token amplifier. Plus a loom-style or exhaustive model check of the wakeup word. 0 **No guest needed, and that is the point** — this is the one crate whose correctness cannot be established by booting.

P2 — GSP boot to INIT_DONE

Build: the boot FSM (GFW progress, falcon/RISC-V CPUCTL, SEC2 booter, WPR2); msgq geometry; ElementLayout as a **descriptor** covering 580 and 610 framing. **Gate:** a stock guest driver reaches GSP_INIT_DONE and proceeds to its first RPC. □ **Carry the version axis from day one.** The 48-byte/16-byte element header split is the cheapest possible proof the descriptor approach works, and retrofitting it later means retrofitting the whole plane.

P3 — the RPC and object plane

Build: the envelope; the 17 functions of `THE_SURFACE_v3.md §1.3`; the object graph; `GSP_RM_ALLOC / GSP_RM_CONTROL` dispatch; the static-info and device-info tables. **Gate:** `nvdev nvidia-smi` in the guest reports the GPU correctly — name, memory, bus info. Binary, externally meaningful, and it exercises ~40 control commands without needing a channel. $\emptyset$ Not a proxy: today `GPU_GET_NAME_STRING` returns 23 zero bytes and `nvidia-smi` prints `ERR!`.

P4 — memory and the VA plane

Build: the single store (one reserved device-local object, guest FB offset = offset into it); the VA manager's mmap/munmap diff, batched with the TLB-defer flag and **one refresh last**; the PTX page-table walk; BAR1/BAR2 windows. $\emptyset\emptyset$ **With the split from `THE_OGKM_RESIDUE.md §3`, in the CORRECTED direction:** the guest owns **PDE3[0]**, we own **PDE3[1]**, and our **BAR2 table is the one hardware walks**. We allocate both root pages in reserved FB and **declare their addresses as `bar1PdeBase / bar2PdeBase` in `GET_GSP_STATIC_INFO`**; the guest then adopts them and writes PDEs into tables we own. $\emptyset$ **BAR1 has no RPC** — the guest writes our page directly — and **nobody sparsifies BAR1 unless we do**. Build it split from the start; a flat model is a rewrite, not a patch. $\square$ **Prerequisite inside P4:** the **guest FB layout must be declared before static info can be served** — root pages, WPR2/FRTS, reserved rows. Today one of those is a captured byte and `bar2PdeBase` is deliberately not written. = **That declaration is the first task of P4, not a detail of it.** **Gate:** `--alias-two-vas, --alias-unmap-observe, --map-propagation, --late-map-race, --missing-page-fault, --uvm-invalidate`.

P5 — channels and the doorbell

Build: channel birth **at alloc** (RM zeroes a caller-supplied `USERD`, so lazy birth wipes the cursor); passthrough doorbell inline on the vCPU; the claim protocol; `Regs/usermode` object handling; the Hopper `doorbell_bar` descriptor field. **Gate:** `--guest-ring-channel, --dictated-ring, --dictated-ring-negative, --doorbell-census, --concurrency, --concurrent-fuzz`.

P6 — the data plane, and the defect that is waiting there

Build: $\square$ **Translated channels — the kind that was never built.** Read `USERD`, advance `GP_GET`, bounded-copy the pushbuffer, rewrite operand apertures into our VA space, submit on our host channel, translate the completion **in address only**. Real CE forwarding. The scrub. **Gate:** $\emptyset$ **the hardware gate, per token:** `forwarded= nonzero` for the kernel scrub tokens (`0x00010001, 0x00010004`), which are `forwarded=0` today. Plus `--ce-client, --ce-client-guest-ram, --cross-client-leak`. $\square\square\square$ **This phase closes the live security defect:** the scrub is currently emulated on the CPU with **no completion writer**, which is the cross-client leak.

P7 — compute

Build: GR routing to real hardware; `GPU_PROMOTE_CTX`; the `ctx-buffer` verbs (`GR_GET_CTX_BUFFER_INFO, KGR_GET_CTX_BUFFER_PTES` — both **value-used and reaching us**). **Gate:** `cuCtxCreate` $\rightarrow$ `2048^2` matmul at `bad=0 maxerr=0, in-guest`.

P4.5 — reset, teardown and re-init. $\emptyset$ Not optional, and it blocks the suite being trusted.

$\emptyset\emptyset\emptyset$ **The stock driver refuses to boot when WPR2 is up.** Clean shutdown runs `unload`; **a guest crash, `reboot -f`, `kexec` and a QEMU `system_reset` do not.** = Without this phase the second boot in a process is a different, worse product than the first. **Build:** tear down every host twin ($\square$ a still-scheduled host channel whose `GPFIFO/USERD` live in guest RAM the new kernel is about to reuse is **a live DMA engine over reused memory**); unmap mirrors; clear the token table; un-poison the register ring; drop the BAR memslots; quiesce workers; reset the boot FSM including WPR2. **Gate:** $\square$ **every earlier phase's gate, run TWICE in one process.** [§6.1]

P5.5 — error paths: RC, host failure, and what the guest is told

Build: the twin state machine — host RM will **RC our channel** on a fault whether we want it or not, and there is currently no path from *"twin dead"* $\rightarrow$ `RC_TRIGGERED` (`fn 0x1004`) $\rightarrow$ the guest's own recovery (`RESET_CHANNEL, TSG preempt, re-alloc`) $\rightarrow$ `re-adopt`. Plus a policy for a host error that arrives **after** the asynchronous reply (`NV_ERR_NO_MEMORY, channel exhaustion` on a shared host GPU). **Gate:** an injected host-side channel fault produces a guest that recovers rather than hangs. $\emptyset$ **And never emit `TRIGGER_BUGCHECK` (`THE_OGKM_RESIDUE.md §5`).**

P8 — deletions

$\emptyset$ **Only now**, and licensed by the **suite**, not by one workload (§8).

3. Ordering constraints — where getting it wrong forces a rewrite

must come first	because
P0 before everything	a gate invented after the fact is a gate shaped to the result
P1 before any guest work	the trap path is the only crate whose correctness a boot cannot establish, and everything runs on it
the split BAR1/BAR2 (P4) before channels (P5)	channels bind VA spaces; a flat model means re-deriving every binding
the version descriptor (P2) before the RPC plane (P3)	framing is per- <code>dg</code> ; a single-version parser is a rewrite of the plane
Translated (P6) before the scrub is trusted	the scrub is a <i>translated</i> channel; emulating it on the CPU is the defect
generated <code>kf-abi</code> (P0/P1) before <code>kf-rm</code> (P3)	9 k lines of hand-written constants is how the current 43 k happened

⊗ **And one anti-constraint:** do **not** build a no-GSP path early “to keep options open”. It has a different oracle, a different floor and a different failure mode, and letting it shape `kf-trap` or `kf-chip` now is how both end up worse.

4. Is 50 k plausible? — honestly

□ **Yes, but only because of generation, and the margin is thin.** Today: 144 175 lines of code across `crates/*/src`. The reduction comes from four places, in order of size:

source of reduction	~lines
<code>kayfabe-isolate-host</code> + <code>kayfabe-isolate</code> deleted (isolates, IPC, fd passing, Wedged)	−52 700
<code>kayfabe-abi</code> hand-written → generated	−30 000
the address table, joins, VA→phys translation, publication epochs, the dirty gate, sweep-skip	−15 000
<code>Proc</code> , the per- <code>proc</code> container, the CPU CE executor, the completion watch, PTIMER	−10 000

4.1 ⊗⊗⊗ An adversarial re-count says 50 k is wrong. I think it is right, and here is the gap.

[w821] A careful re-count puts the realistic figure at **70-90 k Rust + ~5 k C**, on the grounds that the RM plane alone (`abi` + `device` + `rmrpc` + `gsp` ≈ 46 k today) **grows with `dg` coverage**, that the QEMU boundary (~19 k) is deferred by §10, and that nine things are new with no existing code — `Translated`, the drainer/ring, the register generator, reset, RC/fault delivery, four GMMU formats, the Hopper BAR1 doorbell, the read plane, `irqfd`.

□ **I think that count is right about the work and wrong about the measure**, and the difference is worth stating because the owner set the 50 k target:

- □ **It is right that the RM plane dominates and that nine things are new.** Both go into §4’s budget as risk, and P4.5/P5.5 above now exist because of it.
- ⊗ **It counts the QEMU boundary at today’s 19 k.** That number is what it is *because* of the isolate plane, the fd passing and the mirror machinery §8 deletes; `kf-qemu` + `kf-core` are budgeted at 3 k because the shim’s job after P-1 is *dispatch*, not state.
- ⊗ **It treats `dg` growth as line growth.** That is exactly the thing generation is for: a second driver version should add **descriptor rows**, not code. If it adds code, the descriptor approach has failed and the line count is the least of the problems.

⇒ □ **The honest form of the target:** ~50 k is achievable **for hand-written code** if — and only if — `kf-abi/kf-chip` are genuinely generated and `kf-rm`’s served set stays near 60. Counting generated output, the shim and tests, **70-90 k is the right expectation**. □ Those are not competing numbers; they are different denominators, and the plan should quote **both**.

□ **The number to distrust is `kf-rm` at 9 k.** `THE_SURFACE_v3.md` §2.3 enumerates ~40 served controls, ~135 admitted-but-undispatched, and two rule-based admissions. If the served set has to grow much past 60, that crate is the one that breaks the budget. ⇒ **Track it per phase**, and treat a served-control count rising faster than the arm count as the signal.

5. What this plan does NOT do, said plainly

- **Ø No Windows guest work.** The axis is surveyed; nothing is built for it. The one cheap thing it licenses is the `bGspNocatEnabled` **refusal**, which belongs in P3 as five lines.
 - **Ø No no-GSP plane.** §3's anti-constraint.
 - **Ø No display.** `NVA083_GRID_DISPLAYLESS` is NVIDIA's own answer and it is a later phase.
 - **Ø No multi-GPU.** □ But `kf-chip` and the object graph must not *assume* one GPU — the cheapest version of that is to key by GPU from the start and never special-case the single case.
 - □ **Teardown, guest reboot and suspend/resume are not phased above.** They are **not optional** and they are the most likely source of a week-one surprise; see §6.
-

6. Ø The three things most likely to be underestimated

1. **Teardown and re-init.** The current tree has five `RmInitAdapter` cycles per QEMU launch, and WPR2 state that does not reset across boots. ⇒ **Every phase's gate should be run twice in one process**, not once. A plane that works on the first init and not the second is a plane that does not work.
2. **Error propagation to the guest.** Refusals are load-bearing (§1.3's named refusal, the tripwires). But what the *guest* does with `NV_ERR_NOT_SUPPORTED` at each site is modelled nowhere. Ø `0x56` is a status the driver **forgives**, which is exactly why a wrong refusal runs on silently.
3. **Host resource lifetime.** With isolates gone, host RM objects are owned in-process. Guest teardown, guest crash, and VMM exit must each free them — and the host driver is the one party that will not forgive a leak across runs.

Part 3 — The surface, at constant level

STATUS: LIVE, 2026-09-20 (w821). Companion to `THE_ARCHITECTURE_v3.md`. Supersedes the *data* in `THE_SURFACE_v2.md`; that document's KEEP/DELETE *plans* still stand and are kept.

◊ **Written for a reader who does not know NVIDIA's constants.** Every entry gives the constant name, its numeric value, one plain sentence saying what the thing *is*, and what we do with it. □ Every name and number is cited to `research_clones/ogkm` or to this tree. Anything a survey could not establish is marked **[UNVERIFIED]** rather than guessed.

0. The four planes, and why they are different shapes

A guest driver reaches a real GPU through exactly four kinds of traffic. *kayfabe* intercepts all four, and each has a different shape because each has a different **trust** and **ordering** requirement — not because of taste.

plane	transport	who can drive it	our posture
GSP RPC	a command/status ring in guest memory, kicked by a BAR0 register write	guest root (RM)	we are the GSP: we answer, forge or forward
RM control	carried <i>inside</i> GSP RPC GSP_RM_CONTROL, and via <code>ioctl</code> on the host side	guest root (RM)	serve, forward, refuse — by command number
BAR0 MMIO	reads untrapped from a DRAM shadow; writes trap to the vCPU	guest root , except the usermode window	allowlist (§41), shadow write-through
channel / pushbuffer	a GPFIFO ring in guest memory, kicked by a doorbell write	□ unprivileged guest userspace	passthrough / translated / emulated

□□□ **The last row is the one that restructured the design.** The doorbell is the only surface an unprivileged guest process touches directly, which makes it adversarial to the guest's own root — see `THE_ARCHITECTURE_v3.md` §2.1 and `THE_CONSTRAINTS.md` §47.

⇒ Read the four sections below in that order: they go from most-privileged transport to least-privileged, and the machinery gets **weaker and more paranoid** as it goes down.

1. GSP RPC — the plane we answer as the firmware

1.1 What this plane is, for a reader who does not know NVIDIA's constants

Modern NVIDIA GPUs run a firmware processor called the **GSP** (GPU System Processor). On such a part the host driver does not talk to the hardware directly for most things — it sends **messages** to the GSP, and the GSP's own copy of the resource manager (**GSP-RM**) does the privileged work. *kayfabe*'s central trick is that **we are the GSP**: the guest's stock, unpatched driver sends us these messages believing we are firmware.

The transport is a pair of rings in **guest system**, not a register interface:

- a **command queue** the guest writes and we read,
- a **status queue** we write and the guest reads,
- each ring made of fixed-size **elements**, each element carrying a checksum and a sequence number, each message an `rpc_message_header_v03_00` envelope plus a payload.

The guest tells us where those rings are once, at boot, through `MESSAGE_QUEUE_INIT_ARGUMENTS` (*kayfabe-gsp/src/boot.rs*:100, read at :1185). After that the **only** hardware event in the whole plane is a single register write that says “*the command queue moved*” — see §3.

1.2 The structures on the wire

structure	what it is	kayfabe [CODE]	ogkm [ogkm]	axis
msgqTxHeader	the ring's producer header: version, element size/count, write_ptr, flags, and the offsets of the rx header and the element array	TxHeader, kayfabe-gsp/src/ring.rs:131-148 (8×u32, 32 bytes, write_ptr at +16)	msgq_priv.h:37	stable
per-element header	prefixes every element: a checksum and a sequence number, plus transport framing	ElementLayout, kayfabe-gsp/src/element.rs:112-233	message_queue_priv.h:43 (580) / :52 (610)	Ø Dg — BREAKS
rpc_message_header_v03_00	the message envelope inside an element: header_version, signature, length, function, rpc_result, rpc_result_private, sequence — 32 bytes	RpcEnvelope, kayfabe-abi/src/view.rs:558-572	—	Dg (additive)
queue pair geometry	binds the guest's command queue and our status queue, including the swapped rx convention where each side publishes its consumption into the <i>other</i> queue's header	MsgqGeometry, kayfabe-gsp/src/ring.rs:407-570	MSGQ_FLAGS_SWAP_RX	stable

□□□ **The element header is the single best worked example of the Dg axis in this tree**, and it is why §0.2 says descriptors rather than branches:

	580.159.04	610.43.02
header size fields	48 bytes authTagBuffer[16], aadBuffer[16], checksum@32, seqNum@36, elemCount@40	16 bytes mctpHeader, nvdmHeader, checksum@8, seqNum@12

⇒ Size *and* every field offset changed between two versions we must both support, and the elemCount field **disappeared**. kayfabe absorbs this as data — ElementLayout carries elem_count_off: Option<usize> and a TransportHdr that is None at 580 and Mctp{..} at 610 — not as a version branch. **[PROPOSE]** every other axis-carrying fact should take this shape.

1.3 The functions

Dispatch: FunctionCodes::classify(), kayfabe-gsp/src/rpc.rs:211-232. The number↔name mapping is **generated**, not transcribed: kayfabe-abi/src/generated/rpc.rs.

□ [ogkm] The enum (src/nvidia/inc/kernel/vgpu/rpc_global_enums.h) is **byte-identical between 580.159.04 and 610.43.02** for every id we use. 610 only *appends*. ⇒ On this plane the *numbers* are Dg-stable and only the *framing* moves.

#	NV_VGPU_MSG_FUNCTION_	what it is, in plain language	we do	[CODE]
1	SET_GUEST_SYSTEM_INFO	first message after boot: the guest states its driver version, page size and OS, and expects the agreed protocol version back	SERVE	kayfabe-device/src/guestsysinfo.rs:112
64	SET_GUEST_SYSTEM_INFO_EXT	follow-on carrying extra system strings; the real driver returns its status, so it must succeed	SERVE (NV_OK, empty body)	guestsysinfo.rs:122
65	GET_GSP_STATIC_INFO	the guest asks for the big fixed-fact table about this GPU — memory layout, engine list, BAR sizes — before anything else can initialise	SERVE from our device-data model	kayfabe-device/src/staticinfo.rs:182
70	UPDATE_BAR_PDE	the guest hands us the physical root of its BAR2 page directory. □ Only the firmware’s directory is hardware-rooted, so this is the only fact we get about that tree	SERVE (latch it)	kayfabe-device/src/bar2.rs:299
72	GSP_SET_SYSTEM_INFO	a system-description blob (PCI topology and friends), sent early, fire-and-forget	IGNORE — ∅ <i>no reply at all</i> ; echoing would desync the guest’s sequence counter	kayfabe-gsp/src/rpc.rs:318
73	SET_REGISTRY	the guest uploads its entire registry / module-parameter table	IGNORE (no reply)	same
10	FREE	destroy one object handle, or a whole client namespace — RM’s teardown stream	SERVE from the local object graph	kayfabe-rmrpc/src/lib.rs:1095, :1955
21	DUP_OBJECT	alias one client’s object into another client’s handle namespace. ∅ It aliases, refcount++ ; it does not copy — this is the normal UVM flow	SERVE (a graph edge)	lib.rs:1097, :1918
103	GSP_RM_ALLOC	the generic “ <i>create an RM object of this class</i> ” envelope. Nearly every object the guest makes — clients, devices, VA spaces, channels, memory, contexts — rides inside	MIXED, default REFUSE	lib.rs:1229-1276

#	NV_VGPU_MSG_FUNCTION_	what it is, in plain language	we do	[CODE]
76	GSP_RM_CONTROL	the generic “ <i>run this RM control command</i> ” envelope. Almost every specific query or operation rides inside as a nested NVxxx_CTRL_CMD_* id + params — see §2	MIXED, default REFUSE	lib.rs:1521-1572
71	CONTINUATION_RECORD	∅ not a function: an overflow <i>fragment</i> of a message too large for one element. ∅∅ [CORRECTED w821] “only <i>GSP_RM_CONTROL fragments this way</i>” is false — there are three producers: GSP_RM_CONTROL, SET_REGISTRY (via the <i>async</i>, no-wait path) and ALLOC_MEMORY (page-table descriptor RPC). This tree’s own reasm.rs:12-31 lists all three, while its reassembly (:341) handles only the control case. ∅ GSP_RM_ALLOC genuinely does not fragment — it returns	folded into reassembly; standalone is refused	kayfabe-rmrpc/src/reasm.rs:250
47	UNLOADING_GUEST_DRIVER	NV_ERR_BUFFER_T00_SMALL the guest is about to rmmmod. □ Synchronous — the guest blocks until we reply	SERVE (ack-only) , reply mandatory	kayfabe-device/src/inert.rs:119
202	ECC_NOTIFIER_WRITE_ACK	the guest’s own acknowledgement that it finished writing the ECC notifier buffer	IGNORE	rpc.rs:320
228	INIT_GSP_TRACE_CRASH_BUFFER	the guest hands us a {physical address, size} for a buffer the firmware is meant to fill with crash-trace records	SERVE (ack, no-op) — we write no trace records, so it stays zeroed	inert.rs:119
0x1001	GSP_INIT_DONE (<i>event</i>)	we tell the guest “ <i>boot finished</i> ”; its boot poll waits on this	outbound only	kayfabe-gsp boot FSM
0x1003	POST_EVENT (<i>event</i>)	the generic asynchronous-completion carrier firmware uses to notify the guest	outbound only	—

#	NV_VGPU_MSG_FUNCTION_	what it is, in plain language	we do	[CODE]
0x1004	RC_TRIGGERED (<i>event</i>)	we tell the guest a channel or engine was torn down by robust-channel fault recovery	outbound only	kayfabe-gsp/src/fault.rs
—	everything else (~210 ids)	any other function or event id	Ø REFUSE by name — NV_ERR_NOT_SUPPORTED (0x56), zeroed body	lib.rs:1146

□□□ **The safe default is a named refusal, never an echo.** A chain of `CommandPolicy` links is tried in order (`PolicyChain::respond`, `kayfabe-gsp/src/boot.rs:596`); whatever no link answers falls through to `GspFsm::answer`'s refusal (`boot.rs:1719-1763`). Ø An `Echo0k` policy exists (`boot.rs:541`) but is a **differential-test fixture reproducing the old C's behaviour** and is **not installed in the production chain**. This matters because `0x56` is a status the guest driver forgives — so a generic-ack fallback would let a wrong configuration run on, undetected, which is precisely the failure this project has measured repeatedly.

1.4 □ The reply is always built from local state

Ø **No RPC is a verbatim forward.** Applying an `RmEvent` to the local object graph (`Gpu::apply`, `kayfabe-core/src/gpu.rs:5582`) can *latch* a pending host-side spawn, resolved immediately after by `materialize_pending` (:5591). Real host ioctls happen **asynchronously as a side effect**, decoupled from the synchronous RPC reply.

□ **That decoupling is load-bearing and was paid for:** it previously ran inline under the rank-0 lock and **crashed QEMU** (`gpu.rs:3462-3471`).

□ **[UNVERIFIED]** whether any individual modelled `GSP_RM_CONTROL` / `GSP_RM_ALLOC` arm blocks on a host call before constructing its reply. The top-level dispatch does not; the ~15 per-control handler files were not exhaustively checked. ⇒ This is the same question as the synchronous-verb list in §2.4, and it is the one the owner is waiting on.

2. RM objects and control commands — the plane that rides inside the RPC

2.1 What this plane is

Everything the guest driver *does* is expressed as two verbs against an object graph:

- **allocate an object of class C under parent P** (`GSP_RM_ALLOC`, §1 fn 103), and
- **run control command X against object O** (`GSP_RM_CONTROL`, §1 fn 76).

The object graph is a strict tree rooted at a **client**: `client` → `device` → `subdevice`, and alongside it VA spaces, channel groups, channels and engine objects. ⇒ Our job on this plane is to **be that graph** — to answer as RM would, from our own model, without a host GPU behind most of it.

□ **Default-deny, with a named refusal.** Both verbs refuse anything without an explicit decoder: `AllocClassNotPermitted` / `ControlNotPermitted` / `UnknownControl`. Ø There is no generic-ack fallback. §1.3 explains why that matters — `0x56` is a status the driver *forgives*.

2.2 The object classes we model

class	NVIDIA name	what it is, in plain language	we do
0x0 / 0x41	NV01_ROOT / NV01_ROOT_CLIENT	the handle namespace everything else lives under — a session root	emulate (graph node)
0x80	NV01_DEVICE_0	one physical GPU, under a client	emulate ; <code>deviceId</code> decoded for multi-GPU routing
0x2080	NV20_SUBDEVICE_0	a named sub-unit of a device (1:1 on a single GPU)	emulate

class	NVIDIA name	what it is, in plain language	we do
0x2081	NV2081_BINAPI	∅ an opaque handle whose controls tunnel whole to firmware, uninterpreted by the kernel	allow; ∅ load-bearing for cuInit
0x79	NV01_EVENT_OS_EVENT	the event object userspace binds to an eventfd for completion wakeups	emulate ; matched by a later POST_EVENT
0x7e	NV01_EVENT_KERNEL_CALLBACK_EX	an event the guest's own kernel RM allocates at adapter init	emulate ; ∅ params (a guest-kernel function pointer) deliberately not decoded
0x90f1	FERMI_VASPACE_A	a GPU virtual address space — the page-table root a channel binds to	emulate and allocate for real on the host
0xa06c	KEPLER_CHANNEL_GROUP_A	a TSG : channels scheduled together, sharing one VA space	emulate ; hVASpace decoded
0x9067	FERMI_CONTEXT_SHARE_A	a subcontext — an indirect handle reaching a VA space via its TSG	emulate ; hVASpace decoded
0xc56f	AMPERE_CHANNEL_GPFIFO_A	the command-ring channel object (§4)	emulate and allocate on host
0xc7c0	AMPERE_COMPUTE_B	the compute engine object	emulate and allocate on host
0xc797	AMPERE_B	a CUDA process binds the 3D/graphics engine object — sibling of compute on the same GR engine	emulate
0xc7b5	AMPERE_DMA_COPY_B	the copy engine object	emulate and allocate on host
0xc561	AMPERE_USERMODE_A	∅ the object whose 64 KiB CPU mapping is the doorbell page (Part 1 §2.1)	allocate on host
0xc574	UVM_CHANNEL_RETAINER	a handle UVM takes on a channel it did not create, to keep it alive	emulate ; ∅ never forwarded
0xc076	GP100_UVM_SW	the software placeholder class UVM binds for fault-cancel (§4.2)	∅∅ REFUSED — [MEASURED] all 4 requests in a boot refused 0x56. ∅ It is on the shared allowlist but not routed through classify() — an admitted-but-unreachable class

∅ **DENIED by name, each with a reason** (capability.rs:1579):

class	name	why refused
0x3f	NV01_MEMORY_LOCAL_PRIVILEGED	privileged video memory
0x71	NV01_MEMORY_SYSTEM_OS_DESCRIPTOR	∅ would hand the host a guest-chosen pointer
0x402c	NV40_I2C	no physical board bus exists. ∅ [UNVERIFIED w821] the claim that “ <i>RM's own source expects this alloc to fail</i> ” does not check out — i2capiConstruct_IMPL returns NV_OK unconditionally. The refusal may still be right; its stated justification is not

Beyond these, ~89 classes are **allowlisted but opaque** — admitted past default-deny and tracked as a graph node with no facts extracted (Turing/Ampere/Ada/Hopper/Blackwell engine and codec classes, NV50_MEMORY_VIRTUAL, GT200_DEBUGGER, fabric classes). □ **[UNVERIFIED]** whether any of them ever reaches a real host allocation.

□ **Axis note.** The host-side class profile is per-architecture: AMPERE_CHANNEL_GPFIFO_A / AMPERE_USERMODE_A / AMPERE_DMA_COPY_B on GA10x and AD10x, HOPPER_* equivalents on GH100. ∅ **Only the GA10x profile is validated on real silicon.**

2.3 Control commands

Served locally — the ones we answer from our own model, no host GPU touched:

cmd	NVIDIA constant	what it asks
0x20800a36	INTERNAL_GPU_GET_CHIP_INFO	static chip identity — boot registers, architecture, implementation
0x20800a40	INTERNAL_GET_DEVICE_INFO_TABLE	the per-engine device/instance table
0x20800a41	GET_USER_REGISTER_ACCESS_MAP	□ the bitmap of BAR0 registers userspace may touch — directly relevant to Part 1 §2.1
0x20800a4c	GPU_GET_SMC_MODE	whether MIG partitioning is on
0x20800aac	BIF_GET_STATIC_INFO	static PCIe info
0x20800af3	CONF_COMPUTE_GET_STATIC_INFO	confidential-computing capabilities
0x20800a59	GMMU_GET_STATIC_INFO	MMU geometry
0x20800a61	FIFO_GET_NUM_CHANNELS	how many channels this GPU supports
0x20802a08	CE_GET_FAULT_METHOD_BUFFER_SIZE	□ the size RM will DMA into. An empty capture row decoded this as 0 against a real 20480 — a buffer overrun with a hardware writer
0x2080012b	GPU_PROMOTE_CTX	the guest publishing {VA, PA, size, attr} bindings for channel context buffers
0x00801813/4	DMA_SET/UNSET_PAGE_DIRECTORY	bind or revoke a VA space's page directory on a GPU
0x90f10106	VASPACE_COPY_SERVER_RESERVED_PDES	□ the guest handing us real PDE physical addresses for our reserved window
0x20800a9f	GMMU_COPY_RESERVED_SPLIT_GVASPACE_PDES_TO_SERVER	the same, for a GPU-group-global VA space
0xa06f0103 / 0xa06c0101	GPFIFO_SCHEDULE (channel / TSG)	start scheduling this channel, or this whole group
0xa06f0104	NVA06F_CTRL_CMD_BIND	bind a channel to an engine
0x20801210	GR_SET_CTXSW_PREEMPTION_MODE	set graphics context-switch preemption mode
0x20801702	MC_SERVICE_INTERRUPTS	the guest's interrupt-poll bottom half. □ deliberately refused (0x56) to cancel the polling loop
0xa06c0105	NVA06C_CTRL_CMD_PREEMPT	preempt a channel group. □ decided, not echoed — NV_OK only if no member has a live host twin
0x20808159, 0x20808162, 0x20809001/9/64	<i>unnamed GSS-legacy</i>	∅ no open-source symbol exists. Opaque blobs cuInit/cudart demand; answered from measured values. Their categories are GPU_/CLK_/PERF_LEGACY_NON_PRIVILEGED (ctrl2080base.h)
0x20802209	NV2080_CTRL_CMD_RC_GET_WATCHDOG_INFO	the robust-channel recovery watchdog's state. ∅∅ [CORRECTED w821] this was filed as " <i>unnamed GSS-legacy, no symbol exists</i> " — wrong twice : it is named in ctrl2080rc.h:179 <i>and</i> in this tree's own allowlist (capability.rs:811), and bit 15 of 0x2209 is clear so the GSS-legacy rule would not admit it anyway

Forwarded to a real host ioctl — $\emptyset$ exactly one confirmed live path:

cmd	constant	note
0x906f0106	GET_MMU_FAULT_INFO	scoped to a channel's host twin, one ioctl per ask
0x2080a026/84/97	unresolved 0x2080a0xx family	path built but gated off by default = behaves as REFUSE today

□ **[UNVERIFIED]** classify_control/forward_control exist as a general mechanism but have **one call site**. Whether a broader forwarding path exists elsewhere was not established.

Refused by name, with a reason (capability.rs:1506): GPU_EXEC_REG_OPS (0x20800122) and the perf-counter EXEC_REG_OPS (0xb0cc010a) — arbitrary register peek/poke; ALLOC_PMA_STREAM (0xb0cc0105) — hardware performance counters; the SM-debugger trio DEBUG_SET_MODE_MMU_DEBUG / SUSPEND_CONTEXT / RESUME_CONTEXT (0x83de0307/17/18); GPU_REPORT_NON_REPLAYABLE_FAULT (0x20800177) — the fault mechanism is not modelled; and the fabric/NVLink family (0x00e00102, 0x00f10003, 0x20803083).

Admitted but undispatched — ~135 commands pass the allowlist with no handler and fall to the unserviced ledger, returning NV_ERR_NOT_SUPPORTED. They cluster in: GPU/system enumeration (GET_ATTACHED_IDS, GET_PCI_INFO, GET_UUID_FROM_GPU_ID, P2P caps), capability queries (GR_GET_CAPS, FB_GET_CAPS, FIFO_GET_CAPS, HOST_GET_CAPS, DMA_GET_CAPS), descriptive subdevice queries (GPU_GET_INFO_V2, GPU_GET_NAME_STRING, GR_GET_INFO, BUS_GET_PCI_INFO, MC_GET_ARCH_INFO, TIMER_GET_TIME), and third-party-P2P / ZBC / debugger families.

$\emptyset$ **[RESOLVED w821 — and the resolution is that there was no contradiction.]** A first pass reported “*nine decoder modules with no confirmed dispatch site*” and speculated that either a second dispatcher existed or we had built decoders for controls we answer “*not supported*” to. □ **The second dispatcher exists:** kayfabe-device/src/innitables.rs (WantedTable, command map at :1146), and **all ten** are wired through it — eventnotify, businfo, gpu-atomics, fbinfo, cecaps, cepce, grfsinfo, gspfeatures, c2cinfo, pcibars, including EVENT_SET_NOTIFICATION, which the first pass called the sharpest instance. □ **The lesson is about the instrument, not the tree:** “*not in the dispatch table I read*” was reported as “*not dispatched*”. A negative from one grep is a statement about the grep.

□ **Two rule-based admissions that are not enumerable as rows:** any control with the **GSS-legacy mask** bit 15 set (ported from gVisor's nvproxy), and any control on a NV2081_BINAPI object. = Both admit *classes of number*, not numbers. □ An allowlist that admits by rule cannot be audited by listing it.

2.4 □□□□ The synchronous verbs — the list the trap path is judged against

This is the answer to the open question in Part 1 §10.2. “*No RM verb on a vCPU*” cannot be checked until we know which verbs the guest issues and then **immediately consumes a value from**. A verb whose caller reads only a status can be deferred; a verb whose caller reads a **value** it then uses cannot.

○○○ **[CORRECTED w821 — the first version of this section asked the wrong question, and the answer it gave was therefore true of the wrong population.]** I listed what the guest driver's callers consume and then marked each “*served / unserviced*” by us. But most of these controls **never reach us at all**: a control is handled by the guest's **own CPU-RM** unless it is flagged ROUTE_TO_PHYSICAL or its handler issues an explicit RPC. = “*unserviced*” named a gap for verbs we are never asked about.

The table below therefore carries the question that actually matters first: **does it reach us?**

verb	reaches us?	value the caller consumes	our obligation
GPU_PROMOTE_CTX 0x2080012b	□ ROUTE_TO_PHYSICAL	$\emptyset$ none — status-only	answer status
CE_GET_CE_PCE_MASK 0x20802a02	□ ROUTE_TO_PHYSICAL	pceMask	□ served
DMA_SET_PAGE_DIRECTORY 0x00801813	□ explicit RPC	$\emptyset$ none — status-only	answer status
□ GR_GET_CTX_BUFFER_INFO 0x20801219	□ explicit RPC	physAddr, size, aperture, pageSize, bIsContiguous	$\emptyset$ [ADDED w821] value-used and reaches us
□ KGR_GET_CTX_BUFFER_PTES 0x20800a28	□ explicit RPC	the PTEs themselves	$\emptyset$ [ADDED w821] value-used and reaches us
FB_GET_INFO_V2 0x20801303	$\emptyset$ only for indices CPU-RM cannot compute	heapSize, heapStart, ...	□ the four indices UVM asks have local handlers — we answer a <i>different</i> caller

verb	reaches us?	value the caller consumes	our obligation
FERMI_VASPACE_A alloc	□ alloc RPC	∅ status only — □ vaBase/vaSize are overwritten from the local VAS after the call	answer status
GPU_GET_ENGINES	∅ no — local	engineCount, engineList[]	∅ not our gap
GPU_GET_GID_INFO	∅ no — local	GUID bytes	∅ not our gap
FIFO_GET_CHANNELLIST 0x0080170d	∅ no — pure local (chid = pKernelChannel->ChID)	hwChannelId	∅ not our gap
CE_GET_CAPS 0x20802a01	∅ no — local kceGetDeviceCaps	capsTbl	∅ not our gap. (_v2 is 0x20802a03)
GPU_GET_MAX_SUPPORTED_PAGE_SIZE	∅ no — local	page size	∅ not our gap
FB_GET_FB_REGION_INFO	∅ no — local	region limits	∅ not our gap
FAULTBUFFER_GET_SIZE	∅ no — local	buffer size	∅ not our gap

□□□ The two status-only verdicts hold — and they are the result worth keeping

Re-verified at **every** issuer, not just the one originally cited:

- **DMA_SET_PAGE_DIRECTORY** — the caller branches on status and returns `memdescGetPhysAddr(vaspaceGetPageDirBase(...))`, computed **locally**. The params struct has no output field at all (`ctrl0080dma.h:802`).
- **GPU_PROMOTE_CTX** — the three CPU-RM-internal issuers the first version did not check (`kernel_graphics_object.c:130`, the path a CUDA process actually takes; `kernel_graphics_context.c:2195`; `kernel_falcon.c:273`) read back only `bInitialSize/bufferId`, **fields they set themselves**.

⇒ **[PROPOSE]** the rule stands: *a synchronous verb must be answerable from state we already hold, and no synchronous verb may require a host round trip.*

□ **One caveat the first version missed, and it is not about values.** After `SET_PAGE_DIRECTORY` returns, the guest's CPU-RM **commits the root locally and re-enables that VA space's channels**. So *deferrable* was argued purely from value-consumption; **ordering against the next doorbell is a separate argument**, and this document does not make it. ∅ That is the remaining open half of the question, not the value half.

∅∅∅ The “highest-value gap” was not a gap — retracted

[RETRACTED w821.] The first version of this section called `NVC36F_CTRL_CMD_GPFIFO_GET_WORK_SUBMIT_TOKEN (0xc36f0108)` “*the load-bearing gap in the whole document*” because it is value-used and unserved. ∅ **On our plane it never arrives.**

`kchannelCtrlCmdGpFifoGetWorkSubmitToken_IMPL (kernel_channel.c:3181)` forwards to firmware only when `bIsVgpuRpcNeeded`, which requires `IS_VIRTUAL(pGpu)` — a **vGPU** guest. We present as a **GSP-client** guest, so the guest's own CPU-RM computes the token locally via `kfifoGenerateWorkSubmitTokenHal_*` from `{runlistId, chId}` it already holds. The control carries no `ROUTE_TO_PHYSICAL` flag, and the local generator returns only `NV_OK` OR `NV_ERR_INVALID_STATE` — **never 0x56**.

□ **What the doorbell plane actually depends on** — and this is the better statement of it — is that the guest's own `chId` and `runlistId`, **chosen by its CPU-RM and handed to us at channel allocation**, are the values it will write to the bell. Our obligation is at *alloc* time, not at token-query time.

□ **And it leaves a live question in the tree, not in the design:** `kayfabe-device/src/sweep.rs` attributes a bench-observed 0x56 to `kayfabe` refusing this very RPC. If the control cannot arrive, that attribution is wrong and the 0x56 came from a neighbouring verb — `GPFIFO_SCHEDULE (0xa06f0103)` and `NVA06F_CTRL_CMD_BIND (0xa06f0104)` **do** RPC. ⇒ One recorder `grep` for `fn 76, cmd 0xc36f0108` settles it. **This is the kind of error this document exists to catch: a symptom attributed to the first plausible cause on the same page.**

□ The UVM gap closes — in the negative

[CLOSED w821.] The first version flagged “*the UVM kernel module was not searched at all*” as the largest survey gap. Searched: `grep CTRL_CMD kernel-open/nvidia-uvm/*.c` returns **zero hits**. UVM issues every RM control through `nvUvmInterface*` into `nv_gpu_ops.c`. ⇒ `nv_gpu_ops.c` **is the complete UVM verb list**, and the table above is not missing a hidden set.

□ What UVM consumes on the doorbell path: the token it writes to the bell (locally generated, per the retraction above) and the `dmaAddress` from `SetPageDirectory`, which it uses as the PDB for TLB invalidates — and which comes from the CPU-RM's **local** `memdesc`, consistent with the status-only verdict.

3. BAR0 MMIO — reads from DRAM, writes to the vCPU

3.1 The asymmetry, and why it exists

BAR0 is the GPU’s register aperture: ~16 MiB of address space where each 4-byte slot is a control register. A normal emulated device traps **both** reads and writes, and pays a VM exit for each. kayfabe does not:

- **Ø Reads are not trapped.** They are served out of **ordinary DRAM** the guest reads directly, with **no exit and no code running**.
- **□ Writes trap** into `nvkvm_trap_bar0_write` on the vCPU thread.

The reason is the trap contract: a vCPU inside an MMIO exit is **not preemptible**, so every trapped read is a stall charged to a core of the customer’s VM. A register the guest *polls* — and driver init polls several thousand times — would be thousands of stalls for a value that did not change. ⇒ Put the value in memory and let the guest read it at DRAM speed.

[MEASURED] **3 572 of 4 096 BAR0 pages are backable in 12 contiguous runs** (`kayfabe-device/src/plane.rs:2560`), against a read census of **241 722** BAR0 reads.

3.2 The shadow — two structures, and they are not the same thing

□ This is the part most often got wrong: “*the shadow*” names **two** objects.

	RegPlane	ShadowSink / ShadowSegment
what	the model : the source of truth that answers a <i>trapped</i> access	the DRAM the guest actually reads
[CODE]	<code>kayfabe-device/src/plane.rs:1223</code>	<code>kayfabe-qemu-raw/src/shim_unsafe.rs:1656, :1626</code>
shape	ChipProfile + GspModel + ROM image + two RankedMutex state cells + a dead-page bitmap	a Vec of {off, len, base} contiguous byte runs, linearly scanned
guest touches it?	Ø never	□ directly, with no exit

Ø **It is not a flat offset-indexed array and not a page hash map.** It is a small vector of merged contiguous runs — which is why a write to an offset no segment covers is **silently dropped** rather than faulting (`shim_unsafe.rs:1666-1677`).

How a page becomes readable-from-DRAM. `bar0_backable_runs()` (`plane.rs:2579`) sweeps the whole aperture in 4 KiB units and admits a page only if **every dword in it** is answerable without running code: a provably dead dword, a boot-register constant, a ROM byte, a CPU interrupt-tree leaf, the BAR0-window latch, an MMU-invalidate register, or something the GSP model can project. Adjacent admitted pages merge into runs.

□ **The content is computed, never loaded from a capture.** `bar0_shadow_fill` (`plane.rs:2743`) fills each run from the *same* per-arm sources the trapped read path uses, and only then does the shim attach that memory (`:1713`). ⇒ There is no capture table to expire as a vendor regression, which is the failure mode §0.3 forbids.

Write-through is a single funnel. `RegPlane::shadow_write` (`plane.rs:3144`) is the only producer, called from the BAR0-window latch, the MMU-invalidate latches and trigger, the interrupt-tree republish, and the GSP register-group republish (`:3933-4013`).

□ **[UNVERIFIED — a discrepancy worth someone’s attention, not yet a confirmed defect.]** Under `feature = “host-isolates”` a second mechanism, `Regs::back_bar0_dead_runs` (`kayfabe-qemu-raw/src/shim.rs:17527`), independently installs a read-only KVM memslot over `bar0_backable_runs()` backed by a memfd its own comments describe as “*never written... zero resident pages*” — **without calling `bar0_shadow_fill` first**. Its prose still carries the pre-w563 *dead-pages-only* meaning while calling the post-w563 *widened* range, which also includes non-zero boot registers and ROM. The test at `crates/kayfabe-device/tests/bar0_backable_pages.rs:143` explicitly records that w563 changed the invariant from “*the run answers zero*” to “*the run answers what `bar0_shadow_fill` computes*.” ⇒ Either the two paths are gated to disjoint builds, or one of them serves zeros where the guest expects `NV_PMC_BOOT_0`. ØØ **[CORRECTED w821] The premise that this could not be settled was false.** I wrote that the call ordering “*crosses the FFI boundary into the C shim, which is not in this repository*.” **The shim is in this repository** — `qemu/hw/misc/nvkvm/nvkvm.c` — and it calls `kayfabe_shim_bar0_shadow_fill` before `kayfabe_shim_bar0_shadow_attach`, in that order. ⇒ The question is answerable here; what remains unchecked is whether `back_bar0_dead_runs` is also invoked from that file. □ **An “I could not determine this” that was never attempted is worse than an open question — it closes the question while looking rigorous.**

3.3 The trapped-write register table

`RegPlane::write`, `plane.rs:6473`. First match wins; the order is itself load-bearing (the doorbell is classified **before the plane lock is taken**, because its port may block).

Offsets are GA10x/GA106. **Axis** column: what moves the offset or the meaning.

offset	register (ogkm swref)	what it is, in plain language	what we do	axis
0x001700	NV_PBUS_BAR0_WINDOW	the latch that positions the 1 MiB PRAMIN aperture below. Ø [CORRECTED w821] _BASE is 23:0 with a 16-bit shift ⇒ 64 KiB granularity , not megabyte-aligned; and _TARGET (25:24) additionally selects the aperture — vidmem, coherent or non-coherent system. The window is 1 MiB <i>wide</i> ; it is not positioned in megabytes	store the raw word verbatim (a read-modify-write must see back exactly what it wrote); write through to shadow	A
0xBB0090	NV_VIRTUAL_FUNCTION_ - DOORBELL	□ the bell : a work-submission token saying a command ring has new entries	§2 of Part 1 — the security-critical path. Classified with no lock held	Ø die
0xB830A0	..._PRIV_MMU_INVALIDATE_ - PDB	low/aperture half of <i>which page directory to invalidate</i>	latch only, not acted on	A
0xB830A4	..._PRIV_MMU_INVALIDATE_ - UPPER_PDB	high half of that address	latch only	A
0xB830B0	..._PRIV_MMU_INVALIDATE	□ the TLB-invalidate fire register: the guest writes scope + TRIGGER=1, then spin-polls this same register until TRIGGER reads back 0	decode scope, publish address-space changes here (the guest is quiesced — this is the exact GPU boundary), then clear TRIGGER unconditionally . □ A stuck TRIGGER hangs or resets the guest	A
0xB81000+i*4	..._PRIV_CPU_INTR_LEAF(i)	pending-interrupt bitmap, 32 vectors per leaf, write-1-to-clear	update tree, republish the whole group to the shadow, maybe raise an interrupt. No plane lock	A
0xB81200/1400	..._LEAF_EN_SET/_CLEAR(i)	enable mask for leaf i, set and clear ports	same arm	A
0xB81600/1608/1610	..._TOP(i), _TOP_EN_SET/_CLEAR	top-level pending summary and its enables	same arm	A
0xB81640	..._LEAF_TRIGGER	software interrupt injection (write-only)	same arm. □ tested first , to avoid aliasing with the TOP_EN_CLEAR array bound	A

offset	register (ogkm swref)	what it is, in plain language	what we do	axis
0x110C00+i*8	NV_PGSP_QUEUE_HEAD(i)	the GSP command doorbell : “a new command is in the ring”. This is the whole hardware surface of the RPC plane in \$1	post and return — increment a counter, no lock, no inline drain. [MEASURED] this fixed a 1.79 s vCPU stall (plane.rs:5196). Drained later by a worker, one command at a time	A, see below
0xBB0080/0084	NV_VIRTUAL_FUNCTION_TIME_0/1	the GPU’s free-running nanosecond timer, read by the driver for timeouts	Ø refused by name and counted — accepting a write would put guest and host GPU on different timebases	die
0x118234	NV_PGC6_AON_SECURE_SCRATCH_GROUP_05(0)	firmware-boot-progress scratch byte the driver polls for “ <i>GFW boot complete</i> ”	drive the boot FSM; republish group	A
0x118128	..._GROUP_05_PRIV_LEVEL_MASK	privilege mask that must be fully lowered before the guest trusts the progress byte	same	A
0x110100	NV_PFALCON_FALCON_CPUCTL (GSP)	the bit that starts the GSP microcontroller running	boot FSM; is_startcpu() classifies	A
0x1100F4	..._HWCFG2 (GSP)	reports/gates whether the falcon’s RISC-V core is enabled	boot FSM	A
0x110118	..._DMATRFCMD (GSP)	the ucode-load DMA command/status register	boot FSM	A
0x110040/0044	..._MAILBOX0/1 (GSP)	the two halves of the LibOS boot-args physical address	boot FSM	A
0x110008/0018/001C	..._IRQSTAT/IRQMASK/IRQDEST (GSP)	falcon interrupt status, mask, routing	boot FSM	A
0x110004	..._IRQSCLR (GSP)	write-1-to-clear falcon interrupt status. the guest’s ISR clears here before draining our status queue	boot FSM; is_swgen0_clear() classifies	A
0x111388	NV_PRISCV_RISCV_CPUCTL	RISC-V core control; the ACTIVE/HALTED bits the guest polls for liveness	boot FSM	A
0x111528/152C	NV_PRISCV_RISCV_IRQMASK/IRQDEST	RISC-V interrupt mask and destination, ANDed for stall attribution	boot FSM	A
0x840100	NV_PFALCON_FALCON_CPUCTL (SEC2)	start register of the security co-processor that loads protected firmware	boot FSM	A

offset	register (ogkm swref)	what it is, in plain language	what we do	axis
0x840040	..._MAILBOX0 (SEC2)	the Booter's argument — distinguishes a Load from an Unload of the protected region	boot FSM; <code>is_booter_unload()</code>	A
0x840118	..._DMATRFCMD (SEC2)	SEC2's ucode-load DMA command/status	boot FSM	A
0x1FA824/828	NV_PFB_PRI_MMU_WPR2_-ADDR_LO/HI	base of WPR2 , the write-protected region where GPU firmware lives. <code>!HI</code> != 0 is the guest's " <i>is WPR2 already up</i> " test	boot FSM	0 0 die x dg — see below

0 **Everything not in this table is silently dropped** — counted in `unclaimed_writes`, returning `WriteOutcome::nothing()` (`plane.rs:6733-6737`). That includes writes to boot-register constants and to the ROM window, which are read-only from the guest's side and have no write arm at all.

3.4 000 WPR2 is the worked example of two axes crossing in one register

`gb20x.rs:196-227`: on Blackwell the register **moved** to `0x0088A824/828` (`NV_HUBMMU_PRI_MMU_WPR2_ADDR_LO/HI`) for 610-series drivers — **while still answering the legacy** `0x1FA824/828` **pair for 580-series drivers**. Both offsets decode to the same internal variant.

= **One register, two axes at once**: the **die** moved it, and the **guest driver version** decides which address that guest will use. Neither axis alone explains the behaviour, and a design that modelled only "*per-die offsets*" would answer the wrong address for a 580 guest on a Blackwell part. 0 This is the concrete argument for \$0.2's claim that axes are not independent and a per-die table is not sufficient on its own.

0 Related, and **[UNVERIFIED]**: `NV_PGSP_QUEUE_HEAD` was once believed Turing-only; `gh100.rs:51` records the correction that **GH100's driver does write it**. = An "architecture X doesn't use this register" belief is exactly the kind of claim that expires.

3.5 Whole apertures, not registers

range	name	treatment
0x700000-0x7FFFFFFF	PRAMIN , 1 MiB	one aperture , classified before the register decode. Every dword resolves through the <code>0x1700</code> latch: <code>fb_addr = (base << 16) + window_off</code> , then reads/writes the guest framebuffer store, or refuses by name if untranslatable. 0 A bring-up aperture, not a running path
0x300000-0x3FFFFFFF	PROM/VBIOS , 1 MiB	one range on read only , served from a byte image. 0 No write arm exists at all — writes fall to the dropped path
0xBB0000-0xBBFFFF	NV_VIRTUAL_FUNCTION — the usermode window, 64 KiB	0 not a pass-through aperture — decoded register by register : the doorbell at <code>0xBB0090</code> , the PTIMER halves at <code>0xBB0080/84</code> . Any other offset inside it is dropped

range	name	treatment
0xB80000-0xBAFFFF	NV_VIRTUAL_FUNCTION_PRIV	likewise register-by-register: the CPU interrupt tree at 0xB81xxx and the MMU-invalidate trio at 0xB830A0/A4/B0. 00 [CORRECTED w821] these two rows were one row reading 0xB80000-0xB8FFFF, usermode/VF window, which is the wrong range for both — and it placed the doorbell and PTIMER inside it, contradicting §3.3 of this same document 00 [CORRECTED w821] a first pass claimed “no MSI-X code exists in this tree at all.” False — IrqSpec::Msix (kayfabe-vmm), signal_msix (kayfabe-vmm-qemu/src/host.rs:374), and msix_init in the device model at qemu/hw/misc/nvkm/nvkm.c. ☐ Whether the table sits inside BAR0’s range is still [UNVERIFIED]
MSI-X table / PBA	—	separate PCI BARs, 0 never trapped — ensuring that is a requirement (Part 1 §2)
BAR1 / BAR2	framebuffer / instance windows	

4. Channels and pushbuffers — the plane unprivileged guest code drives

4.1 What a channel is, for a reader who does not know NVIDIA’s constants

A **channel** is one command stream to the GPU. It has three pieces of memory:

- a **pushbuffer** — a byte array of commands (“methods”), written by the guest;
- a **GPFIFO ring** — an array of 8-byte entries, each pointing at a run of pushbuffer;
- **USERD** — a small per-channel block holding the ring cursors GP_PUT (producer, written by the guest) and GP_GET (consumer, advanced by the hardware).

To submit work the guest appends a GPFIFO entry, bumps GP_PUT, and **rings the doorbell**. The hardware’s front-end then reads GP_PUT, fetches entries, and executes the methods.

☐☐☐ GP_PUT lives in USERD, which is mapped to the owning process and nobody else. That is the whole reason a hostile doorbell is harmless on real hardware and dangerous for us: hardware re-reads memory only the owner can write, and we *parse the owner’s pushbuffer*. See Part 1 §2.1.

4.2 The three kinds — and an honest statement of what is built

kind	contract	what it means
Passthrough	RingAndReturn — “no inspection and no work”; may run on the vCPU	the guest’s ring is in real GPU memory; hardware fetches it directly. We never read a byte of it
Translated	ScheduleAndReturn; must not run on the vCPU	every entry’s operands rewritten into our own VA space, then submitted on our host channel. The GPU moves the bytes
Emulated	ScheduleAndReturn; must not run on the vCPU	we implement the function in the VMM

[CODE] GuestChannelKind, kayfabe-core/src/channel_kind.rs:96-158; Contracts at :286-315.

~~000~~ **The classifier is a two-way switch, not a three-way one, and Translated is never constructed.** ProcBoundary::channel_kind() (kayfabe-core/src/project.rs:308-316) is:

```
if self.anchor == SYSTEM_ANCHOR { Emulated } else { Passthrough }
```

⇒ A channel is **Emulated** iff its client namespace is the one reserved *system* component (the guest **kernel**’s own client — UVM, the CeUtils scrubber), and **Passthrough** otherwise. □ **No source location tests a class id to decide the kind**. The class id only matters later, for method decode and engine routing.

⊗ GuestChannelKind::Translated is a fully-typed third variant with complete contracts and exhaustive-match coverage everywhere — and **no production path ever assigns it to a channel**; a Translated birth is refused outright (kayfabe-fwd/src/lib.rs:4324). □ **[CORRECTED w821]** I called channel_kind.rs:355 “a test fixture array”; it is pub const ALL, outside #[cfg(test)]. The substantive claim is unaffected — but the parenthetical was wrong, and it was the kind of detail that makes a reader trust the rest. The design doc that introduced it (the_three_channel_kinds.md, STATUS: DESIGN, w803) says “not yet implemented”, and that is still true. ⇒ **Real kernel-CE work the design says should be Translated is today classified Emulated** and driven through the CPU/host-CE forwarding path in §4.5. □ This is the gap behind the scrub defect in Part 5.

class (name + hex)	what it is	kind today	axis
AMPERE_CHANNEL_GPFIFO_A 0xC56F — user CUDA context	a userspace channel a CUDA process owns	Passthrough	A
AMPERE_CHANNEL_GPFIFO_A 0xC56F — guest-kernel / UVM / CeUtils	a channel the guest kernel driver owns	Emulated (design target: Translated)	A
AMPERE_DMA_COPY_B 0xC7B5	the copy engine class object — does DMA copies and memsets	inherits its channel’s kind	A
AMPERE_COMPUTE_B 0xC7C0	the compute class object — kernel launches	inherits; routed HostGr, real hardware runs it, no in-process executor	A
GP100_UVM_SW 0xC076	⊗ not a real engine . A software placeholder so UVM’s channel allocation succeeds, and a subchannel to hold fault-cancel methods	rides its (kernel) channel → Emulated	stable
VOLTA_CHANNEL_GPFIFO_A 0xC36F, TURING_..., HOPPER_... 0xC86F, BLACKWELL_... 0xC96F	the same channel object, per architecture	kind is per-namespace, not per-class	⊗ A
AMPERE_USERMODE_A 0xC561	⊗ not a channel — the BAR window object that exposes the doorbell register itself (Part 1 §2.1). ⊗⊗ [CORRECTED w821] this row read 0xC361, which is VOLTA_USERMODE_A — contradicting §2.2 of this same document	n/a	A

4.3 The GPFIFO entry — 8 bytes

[CODE] GpfifoEntry, kayfabe-abi/src/submit.rs:1836; decode at :1852. [ogkm] NVC56F_GP_ENTRY0/1, clc56f.h:262-284.

field	bits	meaning
GP_ENTRY0_FETCH	0:0	unconditional / conditional fetch (not modelled)
GP_ENTRY0_GET	31:2	pushbuffer address, bits 31:2
GP_ENTRY1_GET_HI	7:0	address bits 39:32
GP_ENTRY1_LEVEL	9:9	main run, or a subroutine call
GP_ENTRY1_LENGTH	30:10	length in dwords
GP_ENTRY1_SYNC	31:31	PROCEED, OR WAIT — the front-end must drain before fetching

□ LENGTH == 0 is treated as a *control* entry whose low byte is an opcode (GP_ENTRY1_OPCODE, 7:0) rather than an empty run, so GET_HI is not read there. We refuse it rather than decode it (submit.rs:1822). □ **[UNVERIFIED w821]** clc56f.h defines GP_ENTRY1_OPCODE but **does not state the LENGTH == 0 rule**. An earlier version asserted it as fact with a citation that does

not carry it. The behaviour is conservative either way — we refuse rather than act — but the *reason* is inferred, not sourced.

□ **Axis note:** NVC36F (Volta) and NVC56F (Ampere) differ in exactly two bits across the whole entry — C56F **dropped** PRIV (8:8) and **gained** INVAL_SCOPE in MEM_OP_A. ⇒ This encoding is far more A-stable than the register offsets in §3. We model neither bit.

4.4 The method header — one dword

[CODE] method_header_decode, submit.rs:2196. [ogkm] NVC56F_DMA_*, clc56f.h:286-360 — bit-identical to NVC36F_DMA_*.

field	bits	meaning
SEC_OP	31:29	which of 8 encodings this dword is dwords that follow, or the immediate payload itself
METHOD_COUNT / IMMD_DATA	28:16	
METHOD_SUBCHANNEL	15:13	which of 8 bound engine objects this addresses
METHOD_ADDRESS	11:0	□ dword-indexed — multiply by 4 for a byte offset

SEC_OP values: GRP0_USE_TERT(0), INC_METHOD(1), GRP2_USE_TERT(2), NON_INC_METHOD(3), IMMD_DATA_METHOD(4), ONE_INC(5), RESERVED6(6), END_PB_SEGMENT(7).

□□□ **Sizing and decoding are deliberately different jobs.** We *size* (skip correctly past) all eight forms, so the stream never desynchronises; we *decode the payload of* only incrementing runs. ∅ RESERVED6 has no defined format, so it decodes to nothing rather than to a guess. **[PROPOSE]** this split — *always know the length, decode only what you model* — is the right shape for every parser in this system, and it is what lets an unmodelled method be harmless.

4.5 Every pushbuffer method we recognise

method (ogkm constant)	offset	engine	what it does	what we do
NVC56F_SET_OBJECT	0x000	host FIFO	binds a class to a subchannel — this is how the stream says “the next methods are for the copy engine”	bind/clear per-subchannel decode state (ga10x.rs:1762)
NVC56F_SEM_ADDR_LO/_HI, SEM_PAYLOAD_LO/_HI, SEM_EXECUTE	0x5C-0x6C	host FIFO	the channel’s own semaphore release : write a payload to a GPU address when this point in the stream retires. This is how the guest learns work finished	decoded only for OPERATION == RELEASE; observed on the completion queue (ga10x.rs:1160)
NVC56F_MEM_OP_A/B/C/D	0x28-0x34	host FIFO	the TLB invalidate transport, targeted at one page directory	decoded for MMU_TLB_INVALIDATE/_TARGETED with PDB_ONE; counted, and the membar honoured as a hard barrier. ∅ no real invalidate is issued (ga10x.rs:1558)
NVC7B5_OFFSET_IN_UPPER/_LOWER, OFFSET_OUT_UPPER/_LOWER	0x400-0x40C	CE	source and destination addresses of a copy	latched (ga10x.rs:1195)
NVC7B5_LINE_LENGTH_IN, LINE_COUNT	0x418,0x41C	CE	how many bytes per line, and how many lines	latched

method (ogkm constant)	offset	engine	what it does	what we do
NVC7B5_SET_SRC/DST_PHYS_MODE_TARGET	0x260-0x264	CE	which physical aperture (coherent system / non-coherent system / vidmem) a physical operand names	latched; read only when that operand is physical
NVC7B5_SET_REMAP_CONST_A/_B, SET_REMAP_COMPONENTS	0x700-0x708	CE	the constant-fill / component-swizzle parameters — this is how a memset is expressed	latched, consumed by remap decode
NVC7B5_SET_SEMAPHORE_A/B, _PAYLOAD[_UPPER]	0x240-0x24C	CE	the copy engine's own completion semaphore address and payload	latched, consumed inside LAUNCH_DMA
NVC7B5_LAUNCH_DMA	0x300	CE	□ starts the copy. Carries no operands — every operand was latched by the methods above; the method word itself is all flags	the whole execute decision (ga10x.rs:1453, kayfabe-fwd/src/lib.rs:8724). See below
NVC076_SET_OBJECT, NO_OPERATION	0x000-0x100	UVM SW	routine bind / no-op on the software class	ordinary; explicitly excluded from the tripwire
NVC076_FAULT_CANCEL_A/B/C, CLEAR_FAULTED_A/B	0x104-0x114	UVM SW	UVM cancelling or clearing a real MMU fault	∅ refuses the whole parse — UvmFaultMethodWithoutFaultDelivery (lib.rs:8919). We have no fault delivery, so seeing one means the assumption behind admitting this class broke. □ A tripwire, not an error
NVC36F_NON_STALL_INTERRUPT	0x020	host FIFO	∅ not recognised anywhere. grep across all crates returns nothing	falls through to opaque
everything else	—	—	WFI, YIELD, NOP, CRC check, sub-device masks, legacy forms, non-incrementing and immediate runs	sized, not decoded — counted as opaque, stream stays in sync

□ **NON_STALL_INTERRUPT being unrecognised is a real gap against Part 1 §5.** That section says the interrupt is *requested in the pushbuffer*, and this is the method that requests it. We currently do not see it. ∅ Today that is survivable only because the channels we care about **poll** — UVM spins, and the scrubber's blocking waits loop on the semaphore word — but it is an assumption about the guest, not a property of our design.

□ **Why LAUNCH_DMA is not in the single-method decoder:** a real copy is **five separate method runs**, and LAUNCH_DMA carries none of the operands. Only the stateful walker that threads an accumulator across the whole run can produce a copy fact (ga10x.rs:1730). **[PROPOSE]** this is the general shape — *a method is not a unit of meaning; a run is* — and any redesign that decodes methods individually will reintroduce this bug.

4.6 What happens when each kind's doorbell rings

Passthrough. The host channel was adopted at **channel allocation**, never lazily (`plan_channel_birth`, `lib.rs:5576`) — mandatory, because RM **zeroes a caller-supplied USERD at alloc time**, so adopting at first doorbell would wipe the cursor that just rang. The doorbell resolves the guest token to the already-adopted host token, runs the ring gate, and writes the real host doorbell. **Ø No pushbuffer byte is ever read** — `device.rs:4110` says so by name, and `ring_content_is_forwardable` returns `false` for this kind. Completions are hardware's own writes to guest memory; we observe nothing and forge nothing.

Emulated. Born lazily at first doorbell if needed. Handed to a worker — must not run on the vCPU. If the bound engine routes to `CpuCe`, the ring is parsed: read the guest's pushbuffer, decode runs into methods, then for each `LAUNCH_DMA` partition the copy into spans by address representability, classify each as a **data copy** (a VA operand) or a **page-table write** (a physical operand — latched and applied to the *owning* process's VA space, which may not be the issuer). Host-CE spans go to the real copy engine and **the real engine writes the guest's completion**. If the engine routes `HostGr`, the doorbell is forwarded but **Ø the ring is never parsed** — “no executor IN THIS PROCESS runs GR work” (`device.rs:8930`).

ØØØ The measured defect, in one line: CPU-executed spans have **zero completion writers** — `write_completion` (`kayfabe-rt/src/cpu_ce.rs:736`; **Ø [CORRECTED w821]** an earlier pointer said `kayfabe-fwd/src/lib.rs:8862`, which is a *comment about it*) has **no callers anywhere in crates/**, pinned by `tests/tests/single_writer_census.rs:360`. A completion on that arm is **silently dropped**. Combined with the scrub being classified `Emulated` rather than `Translated`, this is the cross-client leak in Part 5.

Translated. **Ø** Design only; no doorbell reaches it. The intended path is in `the_three_channel_kinds.md §4`: read `USERD`, advance `GP_GET`, bounded-copy the pushbuffer, rewrite every operand aperture `PHYSICAL → VIRTUAL` into one static VA space, submit on our host channel, and translate the completion **in address only** — the real engine writes the value. An untranslatable operand is meant to be forwarded as a **deliberate fault at a named sentinel VA**, never emulated and never silently dropped. —

5. Ø Where this inventory is knowingly incomplete

ØØØ This section is the point of the document, not an apology. An inventory whose gaps are unmarked reads as complete, and this project's most expensive recurring failure is a plausible value that was never measured. Everything below is a place a survey **stopped**, stated so a reader can argue about it or go and close it.

5.1 Gaps in what we implement

#	gap	why it matters	where
1	Ø Translated is never constructed. The kind exists, is fully typed, has contracts and exhaustive-match coverage, and zero live construction sites	Everything the design says should be <i>translated</i> is today <i>emulated</i> — including the scrub. This is the gap behind the live cross-client leak	§4.2
2	Ø CPU-executed copy spans have no completion writer. <code>write_completion</code> has no call sites	A completion on that arm is silently dropped . A guest waiting on it waits forever or proceeds on stale data	§4.6
3	Ø <code>NVC36F_NON_STALL_INTERRUPT</code> is not recognised anywhere	It is the method by which a guest <i>requests</i> the completion interrupt Part 1 §5 is built around. We survive only because the channels we care about poll — an assumption about the guest, not a property of our design	§4.5

#	gap	why it matters	where
4	∅ No fault delivery exists. <code>NVC076_FAULT_CANCEL_*</code> refuses the whole parse	□ Correctly built as a tripwire rather than an error: it fires exactly when the assumption behind admitting the UVM software class breaks	\$4.5
5	∅ The TLB-invalidate method is decoded and counted, but no invalidate is issued	Modelled structurally with nothing behind it	\$4.5
6	□ Two BAR0 shadow-install mechanisms, possibly overlapping , one of which maps a never-written zero memfd over a range that now includes non-zero boot registers	If both are live, one serves zeros where the guest expects <code>NV_PMC_BOOT_0</code> . Could not be settled — the call ordering crosses the FFI boundary into the C shim, which is not in this repository	\$3.2

5.2 Questions a survey could not answer

#	question	why it was not closed
7	Does any individual <code>GSP_RM_CONTROL / GSP_RM_ALLOC</code> handler block on a host call before replying?	The top-level dispatch does not. The ~15 per-control handler files were not exhaustively checked. □ This is the same question as the synchronous-verb list, and it is the one the owner is waiting on
8	Does the status (reply) queue have its own outbound doorbell, or is it purely guest-pollled?	No explicit statement found; only its absence from the code read. Consistent with the C oracle's <code>IrqRaise == 1</code> with zero interrupt-clear writes, but not proven
9	Is there a top-level RPC function a stock guest sends at init that we fail to classify?	Searched; none found. Absence not proven
10	Does the doorbell token encoding really differ per Blackwell die group?	Carried in this project's notes; not confirmed against <code>gb20x</code> code. The doorbell <i>offset</i> is identical Volta→Blackwell
11	Where does the MSI-X table live, and is it inside BAR0?	∅ No MSI-X code exists in this tree at all — only a vector count. The table and delivery are QEMU's native model on the C side
12	□ <code>sweep.rs</code> attributes a bench 0x56 to kayfabes refusing <code>GPFIFO_GET_WORK_SUBMIT_TOKEN</code> — a control that cannot reach us on a GSP-client guest (\$2.4)	⇒ The attribution is wrong and the 0x56 came from a neighbouring verb (<code>GPFIFO_SCHEDULE / BIND</code> , which do RPC). □ One recorder grep for <code>fn 76, cmd 0xc36f0108</code> settles it
13	□ A too-large SET_REGISTRY would arrive as an ignored fn-73 head plus fn-71 fragments the bridge refuses — i.e. we would <i>reply</i> to a no-wait RPC	∅ Our reassembly handles only the <code>GSP_RM_CONTROL</code> case though three functions fragment (\$1.3). □ Whether a stock guest's registry ever exceeds one element is unmeasured
14	□ <code>GP100_UVM_SW</code> is allowlisted but not routed through <code>classify()</code>	An <i>admitted-but-unreachable</i> class. <code>[MEASURED]</code> all 4 allocs in a boot refused 0x56. ⇒ The allowlist and the dispatcher disagree, and the allowlist is the one that reads as intent

#	question	why it was not closed
15	□ back_bar0_dead_runs vs the C shim's fill-then-attach ordering (§3.2)	Now answerable <i>in this tree</i> — the shim is <code>qemu/hw/misc/nvkm/nvkm.c</code> . What remains is whether that file also invokes the Rust path

5.3 ∅ Axis coverage — the honest statement

Everything in this part was surveyed against **one point of the lattice**:

GPU: Ampere GA106 · **guest OS:** Linux · **guest driver:** 580/610 · **VMM:** QEMU

⇒ The **axis** column in each table says what *should* vary. It does **not** certify that the entry has been exercised at a second point on that axis. Three specific consequences:

- □ **Windows guest has zero coverage.** Every guest-side inference here is Linux-shaped until proven otherwise. The C artifact never ran one either.
- □ **Turing is not covered and is not the easy end.** NVIDIA binds Turing to a *different page-table format family* from the one built here; spanning Turing→Blackwell needs four.
- □ **The cross-axis cases are the dangerous ones.** §3.4's WPR2 register — moved by the **die**, with *which* address in play decided by the **guest driver version** — is the only one this survey found. ∅ It is very unlikely to be the only one that exists.

5.4 What would close these

[PROPOSE], in the order I would do them:

1. **#2 and #1 together** — they are one defect wearing two hats, and they are the live security bug. A completion that is dropped and a channel kind that was never built are both *“the scrub does not really happen.”*
2. **#12** — one grep, and it either confirms an instrument or retires a wrong attribution that is currently sitting in the tree as a comment.
3. **#6 / #15** — the shim is in this repository after all; one read settles whether two shadow installers overlap.
4. **#3** — cheap to decode, and it converts an assumption about guest behaviour into a fact we handle.
5. The axis gaps in §5.3 — □ these are not a task, they are a **standing property** of the product, and they want a second point on the lattice in CI rather than a one-off audit.

5.5 ∅∅∅ What the review of THIS document found, kept as the record

Two adversarial passes over this bundle found **more defects in the document than in the design**, and the pattern is worth stating because it recurs:

- **Two claims were true of the wrong population** — the *“highest-value gap”* that cannot reach us, and *“nine decoders with no dispatch site”* that are all dispatched. Both were built from a correct observation over an incomplete set.
- **One “could not be determined” was never attempted** — the C shim is in this repository. □ That is worse than an open question, because it **closes** the question while looking rigorous.
- **One internal contradiction survived both a table and its own prose** — `0xC361` VS `0xC561` for the same class, in two sections of one file.
- □ **The two headline results survived intact**: both page-directory verbs really are status-only, and the doorbell really is an unprivileged surface. ⇒ The defects clustered in the *supporting* detail, which is exactly where a reader stops checking.

Part 4 — What the oracles tell us

STATUS: LIVE, 2026-09-20 (w821). [owner] “Worth digging *ogkm* source so we get any leftover for either Windows behaviour or non-GSP behaviour, and record it. The rest is *nouveau*.” Surveyed against 610.43.02 with 580.159.04 as the delta reference.

0. 000 THE HEADLINE REFUTES WHAT I TOLD THE OWNER YESTERDAY

THE_WINDOWS_AXIS.md §10.3 argued the no-GSP plane’s real cost is **losing the oracle**, because “*for the native Turing+ path, openrm contains nothing: it is GSP-only.*”

0 **That is wrong, and substantially so.** The drop is *compiled* as a GSP client, but the monolithic material is **present and readable** in three distinct forms. Distinguishing them is the whole skill of reading this tree:

class	what it is	readable?	built?
(1) declarative	the monolithic object model, engine descriptors, module list, the control catalogue	☐ fully	0 #define ... 0
(2) GSP-RM-side code inline in shared functions	if (RMCFG_FEATURE_PLATFORM_GSP) blocks = what the firmware itself does	☐ fully	0 constant-folded
(3) chip-named “monolithic-era” bodies	*_GM107, *_GP100, *_GV100	☐ fully	☐ mostly YES, and mostly LIVE

☐☐☐ **And (3) inverts the intuition that pre-Turing code is dead code.** 56 of 57 pre-Turing-named source files are compiled, and they are the **live base implementations Turing+ inherits**. What is dead is *chip enablement* — every pre-Turing chip has a **zeroed** (arch, impl) row in `g_hal_archimpl.h`, so it can never be matched at probe.

1. ☐☐☐☐ The reading technique — the VF HAL arm is the surviving proxy for monolithic

One example carries the method. `generated/g_kern_bus_nvoc.c:602-617`:

```
if (RmVariantHal: VF) __kbusCommitBar2__ = &kbusCommitBar2_GM107; // the REAL hardware sequence
else __kbusCommitBar2__ = &kbusCommitBar2_KERNEL; // a no-op / an RPC

• kbusCommitBar2_KERNEL (gpu/bus/kern_bus.c:457) does essentially nothing.
• kbusCommitBar2_GM107 (arch/maxwell/kern_bus_gm107.c:5799-5861) is the full physical sequence: rewrite PTEs for the self-mapping pointer and flush page, flush the write-combine buffer, kbusFlush_HAL(BUS_FLUSH_VIDEO_MEMORY), kgmmuInvalidateTlb_HAL(... PTE_DOWNGRADE, NON_LINK_TLBS), swap the PDB, kbusBindBar2_HAL(BAR2_MODE_VIRTUAL).
```

⇒ ☐ **Wherever behaviour is halified on `RmVariantHal`, read the VF arm to learn what the CPU does itself and the `_KERNEL` arm to learn what it delegates.** 376 members are halified this way; 423 generated dispatch sites carry a `RmVariantHal` condition.

☐ The taxonomy is stated by NVIDIA in prose (`generated/g_chips2halspec_nvoc.h:89-94`): “*KERNEL_ONLY: RM does not own HW, the physical part is offloaded to Ucode. MONOLITHIC: RM owns both the interface to the client and the underlying HW.*” 0 But the generated selector has **no arm for PF_MONOLITHIC or UCODE** (`g_chips2halspec_nvoc.c:205-217`) — only VF and PF_KERNEL_ONLY. **That is why the VF arm is the proxy: it is the only surviving “RM owns the hardware” variant.**

2. The four artifacts worth having

2.1 ☐☐☐☐ `inc/kernel/gpu/gpu_child_list.h` — the complete monolithic object model

89 children, 34 enabled, 55 disabled, with construction **order** and a `bConstructEarly` flag. The 55 disabled are exactly the physical-RM object set — OBJFIFO, OBJGR, OBJBUS, OBJGMMU, OBJCE, OBJDISP, OBJVBIO5, OBJACR, OBJVMMU, Pmu, OBJSEC2, ... — declared, readable, not built. ⇒ **Nothing else in the tree tells you what a real RM instantiates, or in what order.**

☐ Corollary the docs never state: in this drop **RM never parses the VBIOS, never runs devinit, never owns ACR/LSFM ucode loading, never owns VMMU, and never owns the runlist scheduler** — all are `RMCFG_MODULE_* = 0`. Every engine exists as a `x / KERNEL_x` pair (33 of them) and in **every pair the physical half is 0**.

2.2 ROUTE_TO_PHYSICAL — a machine-readable manifest of the CPU/GSP boundary

The routing is one function (rmapi/resource.c:254-300): if IS_FW_CLIENT and the flag is set, RPC and **skip the local body**; otherwise **run it**. Measured over the generated method tables:

	count
exported RM control methods	1372
carrying ROUTE_TO_PHYSICAL	753 (54.9 %)
... body stripped (pFunc = NULL)	679
... body retained (also IMPLEMENTED_ON_VGPU_GUEST)	74

⇒ For anything Mode-2-shaped this is the definitive list of **what must be answered rather than executed** — with method id, parameter struct and function name for all 753. And the **74 retained bodies are the physical computation, readable**, including FifoGetDeviceInfoTable (0x20801112), MemSysGetStaticConfig (0x20800a1c), BifGetStaticInfo (0x20800aac), GmmuGetStaticInfo (0x20800a59).

2.3 42 orphaned monolithic bodies — defined once, referenced nowhere

Compiled but unreachable. The highest-value cluster is BAR1/BAR2 bring-up: kbusInitInstBlk_GM107 (kern_bus_gm107.c:5223), kbusBar2InstBlkWrite_GM107 (:5299), kbusBindBar2_GM107 (:5111).

Together they give the **complete instance-block field layout and BOTH write paths**: the bootstrap path pokes NV_RAMIN_ADR_LIMIT_LO/HI and NV_RAMIN_PAGE_DIR_BASE_{TARGET,VOL,LO,HI} through the **BAR0/PRAMIN window**; the steady-state path writes the same fields via a **BAR2 mapping**. ⇒ Directly relevant to this project's PRAMIN and instance-block modelling.

A pre-Turing name proves nothing about liveness. rpcGetEngineUtilizationWrapper_GM204 looks equally orphaned and is wired to **every Turing+ chip** (g_rpc_private.h:4021...5233). **Only the generated tables decide.**

2.4 163 readable if (RMCFG_FEATURE_PLATFORM_GSP) blocks — the firmware's own code

Compiled out, intact. ⇒ The cheapest window into **GSP-RM behaviour** available anywhere, concentrated in mem_mgr/gpu_vaspace.c, mem_mgr/virtual_mem.c, gpu/mem_mgr/heap.c, gpu/gr/kernel_graphics_context.c, rmapi/control.c.

3. The finding that lands on OUR CURRENT DESIGN: BAR1/BAR2 are SPLIT

⊗ An emulator that models one flat BAR2 (or BAR1) VA space will diverge from what the guest expects, and NVIDIA says so twice:

- gpu/bus/arch/turing/kern_bus_tu102.c:215-219 — *"In RM-offload scenario, Kernel RM and Physical RM use their own GPU VA space respectively. The expectation is that the Physical RM base starts from the second PD entry of the topmost PD"* ⇒ mmuFmtEntryIndexVirtAddrLo(pFmt->pRoot, 0, 1).
- mem_mgr/gpu_vaspace.c:255-262, :820-824 — *"BAR1 construction is split between GSP and CPU. Only CPU side performs the sparsification."* And _gvaspaceBar1VaSpaceConstruct/Destruct are **no-ops unless RMCFG_FEATURE_PLATFORM_GSP (:233-237)** — pinning the BAR1 root page directory is **the firmware's job, i.e. ours**.

⊗⊗⊗ [CORRECTED w821 — I stated the ownership BACKWARDS, and the direction is the whole point.] I wrote *"we own PD0[0] and the BAR1 root pin; the guest owns PD0[1]."* **The reverse is true**, per kbusPatchBar2Pdb_GSPCLIENT (gpu/bus/kern_bus.c:816):

"CPU-RM owns the VA range under PDE3[0] and GSP-RM owns the VA range under PDE3[1] ... CPU-RM passes its PDE3[0] value to GSP-RM, then GSP-RM will fill this value to PDE3[0] of GSP-RM's table (only GSP-RM's BAR2 table will be bound to HW)."

⇒ Guest = PDE3[0]. We = PDE3[1]. And OUR table is the one hardware walks. The function literally rewrites the guest's own PDB cache to **our** address: memdescDescribe(pMemDesc, ADDR_FBMEM, pGSCI->bar2PdeBase, ...); pKernelBus->virtualBar2[...].pPDB = pMemDesc;

⇒ Three concrete obligations v3 does not state:

1. **We allocate and bind the REAL BAR2 root**, in reserved framebuffer, and report its address as bar2PdeBase in GET_GSP_STATIC_INFO. The guest then swaps its PDB cache to ours — so **its TLB invalidates name OUR PDB**, and our BAR2 walker roots at our page with **entry 0 = the guest's entryValue** handed to us by UPDATE_BAR_PDE (fn 70). THE_SURFACE_v3.md §1.3's fn-70 row is imprecise: it carries the **PDE3[0] entry**, not a root address.

2. **BAR1 has NO RPC AT ALL.** `kbusPatchBar1Pdb_GSPCLIENT` (`kern_bus.c:766`) makes the guest **adopt our root page at `bar1PdeBase` and write PDEs into it directly.** ⇒ Anything modelling a BAR1 update *message* is modelling something that does not exist.
3. **Nobody sparsifies BAR1 unless we do.** The CPU side skips it for a GSP client (`gpu_vaspace.c:255`), so an unpopulated BAR1 PDE is **sparse, not an error.**

⇒ The real statement of what “*we are the GSP*” obliges here: **we own the root pages of both BARs, we declare their addresses in static info, and the guest writes into tables we allocated.**

3.1 Fault-buffer ownership is the sharpest divergence

	monolithic	GSP client
map the HW fault buffer	into CPU-invisible BAR2	asserted impossible — <code>kgmmuFaultBufferMap</code> has <code>NV_ASSERT_OR_RETURN(!IS_GSP_CLIENT(pGpu), ...)</code> (<code>kern_gmmu.c:2743</code>)
MC_ENGINE_IDX_GMMU, _NON_REPLAYABLE_FAULT, _ERROR	CPU services them (:2278-2299)	never reach the host; “ <i>bounce the interrupt to GSP</i> ” (:2435)
UVM’s fault-buffer GET register	real	“ <i>a dummy register... The real GET register is owned by GSP-RM</i> ” (<code>uvm_gpu_isr.c:693</code>)

⇒ Directly relevant to `THE_ARCHITECTURE_v3.md` §10’s fault-delivery item: **as the GSP we own the PUT pointer and the guest reads it.**

4. Pre-Turing — constants are thin, implementations are LIVE

	finding
swref registers	Fermi has no directory at all; Kepler is ONE 41-line file, included by zero .c. Usable coverage is Maxwell/Pascal/Volta only: 42 headers / 2222 lines
class headers	all present with full method encodings (~36 000 lines Fermi→Volta). <code>GF100_CHANNEL_GPFIFO_0xa06f</code> , <code>KEPLER_CHANNEL_GPFIFO_A_0xa06f</code> , <code>MAXWELL_0xb06f</code> , <code>PASCAL_0xc06f</code> , <code>VOLTA_0xc36f</code> are enabled = 1 and allocatable today
implementations	57 files, 56 compiled, and LIVE as base classes. <code>dmaUpdateVASpace_GF100</code> is the live PTE writer for every supported GPU ; BAR2 bring-up is <code>_GM107</code> for everyone; Ampere’s MMU format literally calls <code>kgmmuFmtInitLevels_GP10X</code>
probe gate	<code>g_hal_archimpl.h</code> retains pre-Turing rows zeroed — { 0x0, 0x0, 0x0 }, // GF100 (disabled). That table is what makes pre-Turing unmatchable
boundary constants	free for the taking: <code>NV_PMC_BOOT_42_ARCHITECTURE_{GM100_0x11 ... TU100_0x16}</code> ; <code>GMMU_FMT_VERSION_{1,2,3}</code> with generation boundaries; <code>NV_RAMIN/NV_RAMUSERD</code> offsets (<code>maxwell/gm107/dev_ram.h:36-67</code>)

⇒ For pre-Turing **REGISTERS** we still need **nouveau** — `ogkm`’s swref stops at Maxwell and has nothing for Fermi/Kepler. ⇒ For pre-Turing **SEMANTICS** `ogkm` is **excellent**, because the code is live and the class encodings are complete. **That is the division of labour the owner asked for.**

4.1 Version rule: use 580 for pre-Turing UVM, 610 for everything else

610 DELETED all pre-Turing UVM material — `uvm_{maxwell,pascal,volta}*` (19 files / 4560 lines), the pre-Turing hwref/ headers (7 files / 2265 lines) and UVM’s local class-header copies. ⇒ **580.159.04 is now the ONLY source for pre-Turing UVM MMU, fault-buffer, CE and host behaviour.** Conversely 610 *added* four pre-Turing swref headers, so **610 is the better source for pre-Turing RM-core registers.** Keep both trees.

5. Windows residue beyond what Part 3 already had

	finding	where
□	GSP can BSOD the host. Event	rpc_global_enums.h:288
□□□□□	TRIGGER_BUGCHECK 0x1023 → osBugCheck(rpc_params->bugCode) (kernel_gsp.c:1748), codes incl. PAGED_SEGMENT 5, BSOD_ON_ASSERT 6. ☐☐ We are the GSP. We must never emit it, and a guest that can induce us to emit it has a guest-crash primitive . □ The code→string table has drifted (g_os_nvoc.h:198-208) — index 5 reads “Invalid Bindata Access”	
□□□□	WDDM creates every address space with MINIMIZE_PTETABLE_SIZE — “to reduce the overhead of private address spaces per application, at the cost of holes in the virtual address space”	nvos.h:1397
□□□□	PTE writes happen at high IRQL on Windows — “This code gets called in a high IRQL path on Windows and shadow buffer allocation may fail there”	gmmu_walk.c:838
□□□□	The GPU lock is released via a DPC at DIRQL — “at high IRQL we can’t signal the semaphore, so we use a second pGpu to schedule a DPC” (three copies)	core/locks.c:1232, 1333, 1419
□□□	FBSR — the only public description of Windows VRAM eviction: pre-reserved pageable sysmem, pinned + DMA’d, or walked in 64 KB chunks through a pinned scratch. MEMDESC_FLAGS_PAGED_SYSMEM is hard-refused on Linux	g_fbsr_nvoc.h:338-347; mem_desc.c:367
□□□	A third runlist policy exists solely for WDDM — ..._CHANNEL_INTERLEAVED_WDDM = 0x2; and Linux RM advertises the WDDM-interleaving cap unconditionally	ctrl2080fifo.h:698; kernel_fifo.c:2842
□□□	UPDATE_PDE_2 requires NVOS32_ALLOC_FLAGS_EXTERNALLY_MANAGED — the WDDM externally-managed-page-table contract, stated in the control’s own doc	ctrl0080dma.h:600
□□	vGPU Windows guests get an 8 KB GSP debug buffer the Linux guest does not; and the emulated guest-OS register is NV_VGPU_GUEST_OS_TYPE {_LINUX 1, _WINDOWS7_DEPRECATED 2, _WINDOWS10 3}	vgpu/rpc.c:539; vgpu/dev_vgpu.h:41-47
☐	Clean negative: WSL, GPU-P, DDA, para-virtualized → ZERO hits . No WSL GPU-paravirtualisation residue anywhere	—

☐ The one Windows × non-GSP statement in the tree: “since this functionality is supported only on Linux,

and we also need support on Windows, **most of the information is collected in physical RM itself**, rather than using a Linux OS layer function" (gpu/disp/kern_disp.c:815-822).

6. ⇒ What this changes

1. ∅ **Retract the §10.3 pessimism.** The no-GSP oracle position is **far better** than “nouveau as a register dictionary”. ogkm supplies the object model, the control-plane manifest, the VF HAL arms, 42 orphaned physical bodies and the class encodings. = **nouveau is needed for pre-Turing REGISTERS; ogkm still carries the SEMANTICS.** □ That does **not** change the priority ruling — GSP stays the stable target — but it materially lowers the no-GSP cost.
2. □ **Adopt the VF-arm reading rule** as standing method, beside the ogkm differential.
3. □ **Keep both driver trees:** 610 for RM-core registers, **580 for pre-Turing UVM**, which 610 deleted.
4. ∅ **Fix the BAR1/BAR2 model** if it assumes one flat VA space — the guest expects the split at PD0[1], and expects us to pin the BAR1 root.
5. ∅ **Never emit** `TRIGGER_BUGCHECK`, and treat any path that could induce it as a guest-crash primitive.

Part 5 — The guest-OS axis

STATUS: LIVE, 2026-09-20 (w821). First survey of the os axis (\$0 of `THE_ARCHITECTURE_v3.md`), which until now carried **zero coverage**. Sources: ogkm 610.43.02 and 580.159.04 (the shared RM core), plus lawfully published Microsoft/NVIDIA documentation, vendor bulletins, open-source projects and conference talks.

∅ **No leaked source or breach material was used.** Two cited items are published security research that reverse-engineers a *shipped binary* (Google Project Zero; a public patcher repo) — legitimate published work, flagged where used. One conference deck was encountered and deliberately **not** relied on; nothing here depends on it.

0. Why this is answerable at all — and exactly where the oracle stops

[owner, 2026-09-20] “NVIDIA open-sourced ogkm under MIT/GPL and still had Windows code in it — that’s so valuable. They could have stripped it with `#if` blocks and then stripped when distributing source for Linux, but they didn’t.”

□ **Correct, and the reason is structural rather than an oversight.** `src/nvidia/` is **one RM codebase compiled for several operating systems**. Stripping per-OS for the public drop would mean maintaining a *divergent* public tree — permanently more expensive than shipping the seams. So the seams ship: **40** `RMCFG_FEATURE_PLATFORM_WINDOWS` sites in 610, **45** `NVOS_IS_WINDOWS`, ~86 WDDM/LDDM comments, and Windows-only control commands documented as such in the SDK headers.

⇒ **ogkm is a genuine partial oracle for Windows behaviour** — the only one that exists, since nouveau and envytools document the hardware and the Linux blob and say nothing about `nvlddmkm`.

∅ **And here is precisely where it stops**, because the distinction decides which claims in this document are sound:

we CAN see	we CANNOT see
□ Policy and decisions in the shared core — which branch is taken on Windows, and why Windows-only control commands and their semantics (<code>UPDATE_PDE_2</code> , <code>RESERVE_ENTRIES</code>)	∅ The Windows OS layer. <code>arch/nvalloc/win/</code> is absent; only <code>unix</code> is published ∅ The Windows rmconfig profile — <code>rmconfig.h</code> hard-sets <code>RMCFG_FEATURE_PLATFORM_WINDOWS 0</code> and <code>NVOS_IS_WINDOWS 0</code> <i>unconditionally</i> , so every NVOC table here is the UNIX binding
Comments stating Windows behaviour in NVIDIA’s own words	∅ Every <code>os*()</code> value on Windows — <code>osGetPageSize</code> , <code>osQueueDpc</code> , <code>osIsRaisedIRQ</code>

□□□ **The `WindowsFirmwarePolicyArg` in §1 is the exact shape of that boundary, in one symbol:** the **parameter** is public and threaded through the shared policy function, and the **struct it points at is not defined anywhere in the published headers**. ⇒ They stripped the OS layer and kept the seams *into* it. We can therefore see **that** a Windows-specific input exists and **where** it applies — and never **what it contains**.

□ The practical rule this imposes on everything below: **a claim sourced from the shared core is strong; a claim about what the Windows OS layer supplies is inference and is marked as such.**

1. ∅∅∅ THE HEADLINE WAS WRONG — GSP is default-ON for Turing+, on every OS

[RETRACTED w821, within the hour, by the owner: “are you sure the stock Windows driver doesn’t turn GSP on when it’s available? On Blackwell it’s even required.”]

The survey reported “GSP is off by default on a stock Windows guest” from community evidence. ∅ **The shared RM core says otherwise, and it is the authority here** — the policy is computed in `src/nvidia/src/kernel/gpu_mgr/`, which is **not** the Unix layer.

```
gpumgrIsDeviceRmFirmwareCapable (gpu_mgr.c):  
  
if (!hypervisorIsVgxHyper() && !_gpumgrIsRmFirmwareCapableChip(pmcBoot42))  
    { bFirmwareCapable = NV_FALSE; goto finish; }    // capable ⇐ arch >= TU100  
...  
if (hypervisorIsVgxHyper()) { ...vGPU-specific chips... }  
else { bEnabledByDefault = NV_TRUE; }    // □ every other case
```

⇒ **For any Turing-or-newer chip outside a vGPU hypervisor, `bEnabledByDefault` is `NV_TRUE`.** □ There is **no OS conditional in this function at all** — the sole platform carve-out is PowerPC. And `gpumgrGetRmFirmwarePolicy` then requests firmware unless the registry *explicitly* says `DISABLED`:


```
*pbRequestFirmware = bFirmwareCapable &&
    (mode == MODE_ENABLED || (bEnableByDefault && mode != MODE_DISABLED));
```

⦿ And the registry key does not mean what the community advice implies

nv-firmware-registry.h: MODE_DISABLED 0x0, MODE_ENABLED 0x1, MODE_DEFAULT 0x2, POLICY_ALLOW_FALLBACK 0x10 — and the **default value is 0x12**, i.e. MODE_DEFAULT | POLICY_ALLOW_FALLBACK.

⇒ Setting EnableGpuFirmware = 1 does **not** flip GSP from off to on. It sets MODE_ENABLED (*force*) and **drops** ALLOW_FALLBACK, because 0x1 does not carry the 0x10 bit.

□□□ **Which supplies a better explanation of the community reports than “off by default”:** GSP was requested, **fell back to monolithic RM**, and setting 1 removed the fallback. The comment in NVIDIA’s own header describes exactly that mode — “*try to enable GPU firmware but fall back if needed... this can result in a mixed mode configuration (ex: GPU0 has firmware enabled, but GPU1 does not).*”

⦿⦿⦿ AND THE RETRACTION ABOVE OVER-CORRECTED — the struct IS defined, and it changes the answer

[CORRECTED AGAIN, same hour.] I wrote that WindowsFirmwarePolicyArg was “*never read in the open tree, and the struct is not defined in any published header.*” ⦿ **The struct is defined**, in generated/g_gpu_mgr_nvoc.h:601:

```
typedef struct WindowsFirmwarePolicyArg {
    NvU32 devId;           // PCI device id
    NvU32 ssId;            // PCI subsystem id
    NvU32 bEnableGpuFirmwareOnWsServerSkus;
    NvBool bIsTccOrMcdm;
} WindowsFirmwarePolicyArg;
```

□□□ **Those four fields say what the Windows default depends on, and it is not uniform:** the **SKU** (devId + ssId), whether this is a **workstation/server SKU** — matching the policy bit NV_REG_ENABLE_GPU_FIRMWARE_POLICY_DEFAULT_ON_WS_SERVER 0x20 in the registry header — and whether the GPU is in **TCC/MCDM** rather than WDDM.

⇒ □ **A per-SKU default with an explicit workstation/server enable is exactly the shape that produces “GSP is on for my Quadro and off for my GeForce.”** The community reports become **plausible again**, for consumer SKUs specifically.

⦿⦿⦿ How I got the retraction wrong — I read a build with the branch compiled out

gpumgrIsDeviceRmFirmwareCapable’s body in this tree **never references** pWinRmFwPolicyArg — it only writes pbEnabledByDefault. I read that body, saw no OS conditional, and concluded “*there is no OS branch in the decision.*”

⦿ **But rmconfig.h hard-sets RMCFG_FEATURE_PLATFORM_WINDOWS 0, so what I read is the Linux-compiled view of a function whose Windows logic is absent from the public drop** — NVIDIA kept the signature and the struct and stripped the body that uses them. □ **The absence of a Windows branch in a build where Windows is disabled is not evidence that no Windows branch exists.** □ Third instance of this tree’s own lesson in one session, in a third disguise: *an empty result is evidence of nothing, not evidence of emptiness.*

□ So what IS established, stated at the right strength

claim	status
Windows runs as a GSP client	□ Proven. RMCFG_FEATURE_PLATFORM_WINDOWS && IS_GSP_CLIENT(pGpu) at gpu_registry.c:224 and gpu_user_shared_data.c:335, selecting a constant named for that configuration : NV_REG_STR_RM_RUSD_POLLING_INTERVAL_WINDOWS_GSP 250 (VS_DEFAULT 500). Plus bGspNocatEnabled, a Windows-only field in the GSP boot RPC itself
The Windows default is per-SKU and per-mode	□ Proven from the struct’s fields — devId, ssId, WS/server, TCC/MCDM
Whether a stock GeForce Windows guest defaults GSP on	⦿ UNRESOLVED — and §1.1 establishes that NOBODY HAS MEASURED IT

claim	status
GSP is required on Blackwell — on Linux	☐ Yes, and the mechanism is in this tree. [owner] “<i>proprietary Linux doesn’t work on Blackwell, so that infers GSP is required anyway.</i>” Confirmed: the open module refuses a non-firmware-capable GPU by name — <code>osapi.c:3721</code> calls <code>gpumgrIsDeviceRmFirmwareCapable</code> and on <code>NV_FALSE</code> prints “<i>installed in this system is not supported by open nvidia.ko</i>”. ⇒ Blackwell + open-modules-only + openrm-is-GSP-only ⇒ GSP required ☐ Does not transfer. <code>nvlddmkm.sys</code> is not the open module, so the Linux chain says nothing about it. ☐ But <code>devId/ssId</code> are policy inputs, so a per-SKU Blackwell default is expressible either way
...and on Windows	

1.1 000 What the public record actually contains — searched w821

[owner] “*maybe find on the internet. GSP Windows Turing+.*” Done. The result is not an answer; it is the discovery that **there is no published answer, and the folklore has no measured basis.**

1. NVIDIA has published nothing about Windows. The authoritative source is the driver README’s GSP chapter, and it is **Linux-only**: “*The GSP firmware will be used by default for all Turing and later GPUs*” — stated for the Linux driver, configured by the **kernel module parameter** `NVreg_EnableGpuFirmware`. ☐ The document **makes no reference to Windows at all**, and NVIDIA documents no Windows registry setting, Control Panel option or app control for it.

2. ☐☐☐ The community claim has no before-state behind it. The primary thread — the one every later citation traces back to — contains **zero documented observations of the default state**. Posters report results *after* setting `EnableGpuFirmware=1`; nobody recorded what `nvidia-smi -q` said **before**. ⇒ “*GSP is off by default on Windows*” is an **inference from the existence of a registry key that people found worth setting**, not a measurement.

3. ☐ And the “N/A means disabled” reading is borrowed from the wrong document. It comes from NVIDIA’s **Linux** README (“*a valid version if GSP firmware is enabled, or N/A if disabled*”) applied to Windows `nvidia-smi`. ☐ That is precisely the misapplication this tree already has a lesson for: an unpopulated field is not a measured zero.

4. ☐ One Blackwell datapoint cuts the other way. A user on an **RTX 5090, driver 581.94** reports a watchdog error that “*immediately stopped after disabling GSP*” — which implies GSP was **on**, and that it is **disableable** on Blackwell/Windows. ☐ Forum-grade, single report, and it does not distinguish “on by default” from “on because they had enabled it”.

5. ☐☐☐ The claim traced to its root — one user, one card, 2023, no vendor reply. The NVIDIA Developer Forums thread every chain leads back to is “*Enable GSP on Windows 11 on my 2080 Ti?*”, **26 November 2023**. The poster observes `GSP Firmware Version : N/A`, finds `gsp_tu10x.bin` in the DriverStore, and writes “*so it is disabled by default, but the driver has GSP binary.*” ☐ **No NVIDIA staff member replied.** One card, no driver version, no before/after, and the conclusion rests entirely on the `N/A` reading. ⇒ **That single post is the origin of the whole claim.**

☐ **But the datapoint is at least ON-POINT, and it is worth saying why.** The RTX 2080 Ti is **TU102 — Turing**, so it sits *inside* the capable set (`arch >= TU100`, \$0.0.1); pre-Turing would be Pascal and earlier. ⇒ The `N/A` is **not** explained away by “this die has no GSP”. ☐☐☐ And the blob the poster found is named `gsp_tu10x.bin` — firmware for *that very chip family*, shipped in the Windows DriverStore. ⇒ The card was capable **and** its firmware was present **and** it still reported nothing. That is the right kind of observation; what it lacks is replication and a known-positive, not relevance.

6. NVIDIA’s ONLY published sentence connecting Windows and GSP is scoped to a different product. In the vGPU user guide’s “*Disabling GSP Firmware*” task: “*For NVIDIA vGPU deployments on Linux and all NVIDIA vGPU software deployments on Windows, omit this task.*” ☐ That is about **NVIDIA vGPU software deployments** — the licensed vGPU/GRID product — not a stock GeForce Windows guest. ☐ It is suggestive (you would not tell people to skip a disable step for something that is on and problematic), but it does not address our configuration and must not be stretched to.

000 And the precise reason the claim is untested: it has no known-positive

The `N/A` inference may well be **right** — `nvidia-smi` is NVIDIA’s own tool and largely shared, so an empty field plausibly does mean “no GSP version to report”. ☐ **It has simply never been checked against a known-positive**: nobody has published a Windows card reporting a GSP version **without** the registry key, on any SKU.

□ And this tree already names that requirement: `a_census_zero_needs_a_known_positive`. A zero from an instrument nobody has seen produce a non-zero, in that configuration, is not a measurement. □ The guru3D posters who set the key *and then saw a version* prove the **instrument works on Windows** — which is exactly what makes the missing observation cheap to take and inexcusable to keep inferring.

⇒ □□□□ **The convergence is the useful part.** The open tree says the Windows default is decided **per-SKU and per-mode** (`devId`, `ssId`, `WS/server`, `TCC/MCDM`). The public record says **nobody has measured it**. Those two facts fit together exactly: *a per-SKU default is what produces contradictory folklore*, because different people are correctly reporting different cards.

□□□ SETTLED w821 — the owner measured it, and the known-positive was already on the record

[owner, 2026-09-20] “Confirmed. On an RTX 1660 Ti Windows PC, GSP firmware is N/A. It’s real.”

□□□ **That closes it, and the reason is worth spelling out** — the measurement that was missing was never the N/A; it was the **known-positive**, and the record already contained one:

leg	evidence
the instrument does report a version on Windows	□ the forum posters who set <code>EnableGpuFirmware=1</code> and then saw a version . ⇒ Windows <code>nvidia-smi</code> populates that field when there is something to report — so an N/A is not an unpopulated field
unmodified consumer Turing reports nothing	RTX 1660 Ti (TU116) [owner, measured] · RTX 2080 Ti (TU102) [forum, 2023] — two independent Turing consumer dies
both are inside the capability set	<code>arch >= TU100</code> (\$0.0.1), and <code>gsp_tu10x.bin</code> ships in the DriverStore

⇒ □ **GSP is off by default on consumer Turing under Windows.** [MEASURED]

□ How far that generalises — three different strengths, and they must not be merged

[owner] “the Windows PC I tested had GSP off on Turing, so GSP off is default under Windows — not entirely true right.” □ **Correct on both halves: the direction is right, the blanket form is not.** Stated at the strength each part actually has:

claim	strength
consumer Turing, Windows ⇒ GSP off by default	□ [MEASURED] — owner’s 1660 Ti, plus the 2080 Ti report, with the known-positive established
not default-on for any consumer SKU on Windows, incl. 50-series	● [FORUM, weak] — two posters assert it and 50-series owners report only post-registry results. ∅ No before-state <code>nvidia-smi</code> on Ada or Blackwell
workstation / server SKUs may differ	□ [STRUCTURAL] — <code>bEnableGpuFirmwareOnWsServerSkus</code> is a field in the policy struct, and <code>POLICY_DEFAULT_ON_WS_SERVER 0x20</code> is a registry bit. ∅ NVIDIA built a switch for this, which is evidence the two classes are not the same
TCC / MCDM	∅ unknown — <code>bIsTccOrMcdm</code> is a policy input, so the mode plainly matters, and nothing measured

⇒ □ **The safe form for design purposes:** “on Windows, assume GSP is OFF unless we observe otherwise, and never assume a single answer across SKUs.” That is stronger than the measurement alone (it covers Ada/Blackwell consumer, on the weak evidence) **and** weaker than a blanket claim (it does not extend to workstation/server or TCC). ∅ A design that hard-codes *either* a Windows-always-GSP or a Windows-never-GSP assumption is wrong — the policy struct says the answer is **per-SKU**, and we must **detect**, not assume (§7’s `bGspNocatEnabled`, and the fallback detector in §1).

□ And a note on how this resolved, because the pattern recurs: I spent three exchanges arguing about the N/A reading when the thing that settled it was **the other leg of the instrument check**. The known-positive existed in the same threads I had already read. ∅ *A census zero needs a known-positive* — and I kept re-examining the zero.

Sources: [NVIDIA 580.65.06 GSP chapter](#) · [guru3D: Enable GSP Firmware on Windows](#) · [guru3D: How to disable NVIDIA’s GSP Firmware on Windows](#) · [Overclock.net: RTX 5090 GSP Firmware](#)

□ **And one thing the “Turing+” framing in that search should not be allowed to blur** [owner: “pre-Turing it can't be used”]: **Turing+ is the HARDWARE capability gate** (`_gpumgrIsRmFirmwareCapableChip`, `arch >= TU100`), not a Windows policy statement. Below Turing there is no GSP to enable on any OS. ⇒ Everything uncertain in this section sits **inside** the capable set, in the *default-on policy* — see `THE_ARCHITECTURE_v3.md §0.0.1`.

⇒ □ **The practical position:** design for GSP being present, **detect and refuse the alternative** (§1's fallback detector), and treat “*stock GeForce Windows defaults GSP off*” as an open risk to be measured — not as a settled premise in either direction.

□□□ How the survey got it wrong, and it is this tree's own documented failure

The community evidence rested partly on `nvidia-smi -q` reporting `GSP Firmware Version : N/A`. ○ **That is not evidence GSP is off. It is evidence the field was not populated.** □ This repo already carries that lesson, paid for once: *an empty capture is evidence of NOTHING, not evidence of emptiness* — the `dlen=0` rows whose every checked instance was **contradicted** by real hardware. ⇒ The survey reproduced the exact error the tree documents, from a different direction.

□□ What replaces it, and it matters on every OS

○○ **The default policy ALLOWS SILENT FALLBACK TO MONOLITHIC RM.** That is 0x12's 0x10 bit, and it is the default on Linux too.

⇒ **If a guest's RM falls back, our fake GSP is never used at all** — monolithic RM drives our emulated registers directly, and every RPC-based assumption in this document evaporates **with no error and no message**. □ **[PRO-POSE]** we must be able to detect that and refuse it by name. The detector is cheap: a guest that has fallen back **never publishes a message queue** and never sends `SET_GUEST_SYSTEM_INFO`, so “*boot progressed past the point where an RPC was due, and no queue exists*” is a nameable, checkable state rather than a hang.

⇒ **Net effect on the Windows question:** §1 no longer argues that the premise fails. It argues that GSP is requested on Turing+ regardless of OS, that a **fallback** path exists and is enabled by default, and that whether NVIDIA's Windows driver takes it is **[UNVERIFIED]** and gated by a struct we cannot see. □ **Sections 2-4 below are unaffected** — they are about WDDM's ownership of the page tables, TDR, and the absence of UVM, none of which depends on how GSP is enabled.

2. Under WDDM the OS owns the page tables — and we never see the writes

Microsoft, verbatim: “*VidMm manages the GPU virtual address space of all processes. VidMm is also responsible for allocating, growing, updating, ensuring residency, and freeing page tables.*” NVIDIA's side matches: VA spaces are `SHARED_MANAGEMENT` (“*management of the VA space is shared with another component (e.g. driver layer, OS...)*”, `nvos.h:1384`), and ranges are `NV0S32_ALLOC_FLAGS_EXTERNALLY_MANAGED` — “*Page tables for this allocation will be managed outside of RM.*”

○○○ **And the transport for individual PDEs is invisible to us.** `NV0080_CTRL_CMD_DMA_UPDATE_PDE_2` (0x80180f) is documented “**only available on Windows and MODS platforms**” and **formats the PDE into the caller's own buffer** rather than RM's directory. Critically, it is **handled purely in guest CPU-RM and is NEVER RPC'd to GSP for a GSP client** (`dma.c:320-324` RPCs only under `IS_VIRTUAL_WITHOUT_SRIOV`; `:340-355` does the work locally). Same for `NV90F1_CTRL_CMD_VASPACE_RESERVE_ENTRIES` (`gpu_vaspace.c:4109`).

⇒ **The address model in Part 1 §4 assumes RM authorship of page tables. On Windows that assumption is false**, and our fake GSP hears nothing.

□ **One thing survives:** `SET_PAGE_DIRECTORY` is RPC'd to GSP when `IS_GSP_CLIENT` (`dma.c:507-521`). □ But with different semantics — aperture typically `_VIDMEM` not `_SYSTEM_COH`, arriving per device creation rather than per UVM VA space.

□□□ **And the silver lining, which is real:** unlike Linux, Windows *tells the driver every PTE it wants written, explicitly and in order*, through `DxgkDdiBuildPagingBuffer` ops (`UpdatePageTable`, `FlushTlb`, `MapApertureSegment`) submitted as **GPU-executed paging buffers**. ⇒ If we can see paging-buffer submissions, that is a **richer** signal than the doorbell-time page-table sweep the C artifact used. ○ **[UNVERIFIED]** whether NVIDIA's WDDM path routes those through CE (visible to us), BAR1 CPU writes, or `UPDATE_PDE_2` (invisible). **This is the single most important open question on this axis.**

3. TDR — a hard, OS-enforced 2-second deadline

	Linux	Windows
graphics timeout	4 s	☐ 2 s , OS-enforced
compute timeout	30 s	2 s (5 s TdrDdiDelay)
consequence of exceeding repeat	the driver soldiers on with stale TLBs —	adapter reset ; VIDEO_TDR_FAILURE (0x116) ☐ five hangs in one minute ⇒ bugcheck 0x117

RM’s own comment concedes it: “90 % of the overall hard limit of **2.0 seconds, imposed by WDDM driver rules**” (os.c:2116).

⇒ ☐ **On Linux a slow emulator is merely slow. On Windows a completion path slower than 2 s is diagnosed as a hung GPU.** ☐ Cold boot is probably safe — device init gets a 60 s exception (thread_state.c:331) — but **runtime is not**, and our data plane is currently a CPU copy loop.

☐ Related and worse: Windows has a **first-class page-fault interrupt** carrying {FaultedVirtualAddress, PageTableLevel, FaultErrorCode, FaultedProcessHandle} and the OS *expects* an accurate fault path, using it to kill the offending context rather than the adapter. ☐ The C artifact went green “*without servicing or forwarding a single GPU fault.*” That is a **Linux-only luxury**.

4. Four more that matter, briefly

- ☐ **No UVM on Windows.** “Currently, we don’t support UVM on Windows/MODS” (pool_alloc.c:488); in WDDM mode CUDA contexts are dxgkrnl contexts (D3DKMT_CLIENTHINT_CUDA). ⇒ The uvm_* ioctl stream **disappears** — and UVM_MAP_EXTERNAL_ALLOCATION is the lockstep boundary of our nvdiff oracle (221 of cuCtxCreate’s 479 ioctls).
- ☐ **The OS manages the framebuffer** on non-TCC Windows — RM verbatim: “Disabling in Non-TCC windows because **the OS manages FB**” = **no scrub-on-free**, and RM stops choosing PTE kinds (bRmToChooseKind = NV_FALSE). ⇒ The scrub path in Part 5 is one a Windows guest largely does not take.
- ☐ **System GPU mappings become 4 KB-granular** where Linux uses 64 KB/2 MB — a 16-512× PTE-count increase. ☐ And Windows **guarantees** VA/PA 64 KB congruence and lets the driver *rely* on it, so a violating emulator faults with no obvious cause — the same class as this tree’s existing congruence trap.
- ☐ **DEV_xxxx must already be in NVIDIA’s signed INF.** PnP binds by hardware id from a catalog-signed package, and “*even a single-byte change... invalidates the digital signature.*” ⇒ **A gate before any of the above matters.**

☐ **And one that lands directly on w821’s Hopper work:** RM allocates the BAR1 doorbell region **downwards from the top of BAR1 on Windows** — “WAR for Bug 3564398, need to allocate doorbell for windows differently” ⇒ BUS_MAP_FB_FLAGS_MAP_DOWNWARDS for MEMDESC_FLAGS_MAP_SYSCOH_OVER_BAR1 (mapping_cpu.c:506), **the exact flag Hopper+ sets on the usermode/doorbell region** (kernel_fifo_gh100.c:123). ⇒ Our BAR1 doorbell-page identification (Part 1 §2.1) must not assume Linux allocation order.

5. ☐☐☐ The escape hatch — TCC, and it collapses most of the table

TCC mode is not WDDM at all — it “uses the Windows WDM driver model” and disables graphics (nvidia-smi -dm 1). ☐ And nearly every Windows branch found in ogkm is gated on !tcc: FBSR-WDDM mode, PMA client page tables, P2P mailbox, scrub-on-free. nv_gpu_ops.c:2327 states it directly: “**Non-TCC mode on Windows implies WDDM mode.**”

⇒ **In TCC: scrub-on-free returns, PMA-managed client page tables return, EXTERNAL_HEAP_CONTROL is off, FBSR-WDDM is off, and dxgkrnl/VidMm/VidSch are entirely out of the loop.** §§2, 4 largely evaporate. ☐ TCC is not available on GeForce parts. (MCDM is the middle option — render-only, no display, and it **requires an MMU**.)

[PROPOSE] If a Windows guest ever becomes a target, **TCC is dramatically the smallest one**, and the order is: TCC → MCDM → WDDM.

6. ☐ What is the SAME — this bounds the work, and it is a lot

1. **The RM API ABI is identical** — NV0521/54/32/33/38, handle semantics, class ids, control numbers. No Windows variants in common/sdk/nvidia/inc/.
2. **NV01_EVENT_WIN32_EVENT is NV01_EVENT_OS_EVENT** — literally a #define, value 0x79.

3. ☐ **The GSP boot sequence is OS-agnostic — zero** Windows conditionals in the bootstrap path; FWSEC/WPR2/booter/LibOS-args/msgq/gsp_init_done all shared. The only platform conditional makes Windows hold the API lock across *all* of GSP init = **strictly more serial, the easier case for us**.
4. **GspSystemInfo layout is identical**; 610 vs 580 differ by four fields — a version difference, not an OS one.
5. **SET_REGISTRY format is the same**; nvrm_registry.h:24 says outright “*shared between Windows and Unix*.”
6. ☐ ☐ **The doorbell / work-submit path has NO Windows conditional in RM** — usermode region, workSubmitToken, kfifoUpdateUsermodeDoorbell_*, USERD, GPFFIFO: **zero** Windows branches across gpu/fifo/. = **Part 1 §2 is OS-independent**, modulo §7’s two unknowns.
7. **GMMU formats, PTE/PDE encodings, apertures, big-page selection** — chip-keyed, not OS-keyed.
8. **Hypervisor detection is shared and CPUID-based**, and **there is no error-43-style refusal anywhere in the open RM core**.
9. **Robust channels / RC are enabled on Windows, GSP and Unix alike**.
10. **PMA is supported on all platforms** — WDDM disables *client page tables under PMA*, not PMA.

7. ☐☐☐ A free gift — detecting a Windows guest from the FIRST RPC

GSP_SET_SYSTEM_INFO carries NvBool bGspNocatEnabled, and the **only** assignment in the entire tree is:

```
if (RMCFG_FEATURE_PLATFORM_WINDOWS) { rpcInfo->bGspNocatEnabled = NV_TRUE; }
```

kernel_gsp.c:4624 (610) / rpc.c:10650 (580) — present and identical in both generations.

⇒ **bGspNocatEnabled == NV_TRUE in the first RPC we ever receive ⇒ the guest RM was built for Windows**. ☐ It is an *enable* flag, not an OS field — **assert on it, do not build on it**. There is no explicit OS identifier in GspSystemInfo.

☐ **[PROPOSE] Implement this now, as a refusal, before any Windows work happens**. It costs one comparison and converts “*a Windows guest silently does something strange*” into a named refusal at the first message — which is exactly what support_matrix_asymmetry demands of an out-of-band configuration.

8. ☐ A correction to this project’s own framing

CLAUDE.md describes nvkvm as “*WSL2-style NVIDIA GPU ioctl/RPC forwarding*.” ☐ Publicly, WSL2’s mechanism is **D3DKMT-shaped thunks over VM Bus to a host dxgkrnl** — the Windows Graphics Kernel lead, XDC 2020: “*Level of abstraction is the WDDM interface... Not a straight pass-through... No data copy — only control information exchanged over VM bus*.” = **Mode 1 resembles WSL2’s shape, not its protocol**. The phrase should be corrected where it appears, because it invites the assumption that a documented design exists for what we do.

☐ Related, and worth knowing: Microsoft’s own answer to our problem statement is **not to run the vendor KMD in the guest at all** — GPU-PV puts “*no KMD in the guest, only UMD... no video memory manager (VidMm) or scheduler (VidSch) in the guest*.” = **The industry cut line is at D3DKMT, not at the hardware**. We are deliberately below it, and that is the thing that makes this project unusual — and the reason no prior art exists for emulating an NVIDIA GPU to a Windows guest.

9. What could NOT be determined, and the first measurements to take

question	status
The Windows OS layer itself (osGetPageSize, osQueueDpc, osIsRaisedIRQL, ...)	☐ arch/nvalloc/win/ is not published. The 4 KB page-size finding rests on an inferred osGetPageSize()==4096
The Windows rmconfig profile — which chips, classes, engines and HAL bindings	☐ openrm hard-sets RMCFG_FEATURE_PLATFORM_WINDOWS 0; every NVOC table here is the UNIX profile
Is the GSP firmware image byte-identical between Windows and Linux packages?	☐ Nothing speaks to it. If it differs, our WPR2/radix3 sizing constants do too
Is the GSP-RM RPC protocol the same on Windows?	☐ inference only (one RM codebase). ☐ Do not assume our msgq work transfers without measuring
What do dxgkrnl/VidMm actually issue, and in what order — CE, BAR1 CPU writes, or UPDATE_PDE_2?	☐ The most important open question on this axis (§2)
Is display mandatory for nvlddmkm to load?	☐ Cosmetic on Linux, plausibly fatal on WDDM. TCC/MCDM sidestep it

question	status
Does NVIDIA enable WDDM 3.2 user-mode doorbells ?	∅ If so, the guest rings a doorbell from user mode with no kernel transition — a trap-per-doorbell design becomes a per-write exit on a user-mode hot path
Was the Code-43 CUID check removed at R465, or gated?	∅ NVIDIA said a narrow configuration is <i>supported</i> ; it never said the check was removed

□ **[PROPOSE]** The first two measurements, in order: 1. `D3DKMT_QUERY_GPUMMU_CAPS` on real hardware — `VirtualAddressBitCount`, `PageTableLevelCount`, 4 K vs 64 K segments, `GpuMmu` vs `IoMmu`. Cheap, and it decides \$2 and the page-size finding. 2. **Check our emulated `DEV_xxxx` is already in NVIDIA's signed INF.** If it is not, nothing else on this axis is reachable with a stock guest.

10. □ Could we support a GSP-DISABLED guest? — nouveau measured, w821

[owner] “Maybe we can wire disabled GSP in kayfabe. Nouveau source has the registers that tell us what NVIDIA writes outside GSP... worth looking from a nouveau clone how no-GSP would look.”

Cloned (`research_clones/nouveau-src`, sparse, 31 MB) and measured.

10.1 □ nouveau IS the right oracle, structurally

□□□ **nouveau implements both paths side by side, and they are separable by filename.** For each subdevice there is a native per-chip file and a GSP-client file:

```
nvkm/subdev/devinit/tu102.c    ← native: drive the registers ourselves
nvkm/subdev/devinit/r535.c     ← GSP: ask the firmware
nvkm/subdev/gsp/rm/r535/*.c    ← the whole GSP-RM client (rpc.c, fifo.c, gr.c, disp.c)
```

⇒ A per-subdevice diff of `tu102.c` against `r535.c` shows **exactly what the driver must do itself when there is no GSP**. That is a real, cheap, repeatable experiment.

10.2 The measurement

	lines of C
nvkm total (the hardware abstraction)	130 118
— subdev/ (init and management)	66 964
— engine/ (runtime)	56 863
the entire GSP subdev , including the RPC client	10 957
native Turing+ per-chip deltas	7 137

□ **The 7 137 is misleading and must not be quoted as the cost.** Turing's native support is a thin delta on an inheritance chain — `fb/tu102.c` pulls `gf100_fb_dtor`, `gm200_fb_init`, `gp100_fb_init_unkn`. ⇒ Emulating a native driver means the **whole accumulated register interface**, not the per-chip delta.

□ **But the honest delta is much smaller than “all of it”, for two reasons:**

- ∅ **The runtime plane is GSP-independent and carries over unchanged.** `fifo` (7 906) and `ce` (756) are hardware, not firmware — consistent with `ogkm`, where `gpu/fifo/` has **no GSP conditional** beyond which side generates the token. ⇒ **Everything in Part 1 \$2 — the doorbell plane, the token table, the trap classifier — is reused as-is.** The delta is **init only**.
- ⦿ Roughly **22 400 lines** of `subdev/` is board and power management — `clk` 6 189, `bios` 8 033, `i2c` 3 058, `therm` 2 614, `volt` 1 023, `gpio` 855, `mxm` 690. An **emulated GPU** has no board to manage; much of this stubs rather than ports.

⇒ **Order-of-magnitude delta: ~30 k lines of register-level init semantics**, of which we already model a slice (`fb`, `instmem`, `bar`, `mmu`, `mc`, `fault`). □ **A line count is a proxy for surface, not a port estimate.**

10.3 000 The real cost is not lines — it is losing the oracle

000 [RETRACTED w821 — the ogkm residue survey refutes this. See Part 4.] The claim below is too strong: ogkm turns out to contain the monolithic object model, a machine-readable manifest of the CPU/physical control boundary, **376 HAL members whose VF arm IS the monolithic body**, and 42 orphaned physical implementations — including the complete BAR1/BAR2 instance-block layout. ⇒ **nouveau is needed for pre-Turing REGISTERS** (ogkm’s swref stops at Maxwell and has nothing for Fermi/Kepler); **ogkm still carries the SEMANTICS**. The cost of the no-GSP plane is materially lower than this section claimed. □ It does **not** change the priority ruling.

□□□□ **nouveau documents the HARDWARE. It does not document NVIDIA’s driver.**

Our entire method is “*be what NVIDIA’s driver expects.*” On the GSP path we have **ogkm — actual NVIDIA source** telling us what the guest will send and what it does with the reply. ∅ **For the native Turing+ path, openrm contains nothing**: it is GSP-only. So a no-GSP mode means emulating what NVIDIA’s **closed monolithic RM** writes, in what order, with nouveau serving only as a *register dictionary*.

□ And nouveau’s native Turing support is **known-incomplete** — no reclocking, signed-firmware limits. = “*nouveau can init Turing natively*” does **not** imply “*nouveau does what NVIDIA’s RM does.*” It is a dictionary, not a behavioural trace.

10.4 What it would buy, stated fairly

- □ A Windows guest with GSP off, **if** that configuration is real (\$1 — still unmeasured).
- □□□ □ **It dissolves the Turing floor.** THE_ARCHITECTURE_v3.md \$0.0.1 argues Turing+ is an *architectural* boundary because below it there is no GSP to impersonate. A no-GSP mode makes **pre-Turing reachable**. ⇒ That is either a significant widening of the product or scope creep, and it is an **owner decision** — flagged because it was probably not the intent of the question.
- □ **It deletes the entire GSP boot fiction** — FWSEC, WPR2, the booter, LibOS args, the msgq, the radix3 ELF, and the 139 821 PROM/VBIOS reads in cap1_coldboot_hermetic. □ That fiction has cost real time (WPR2 state not resetting across boots; five RmInitAdapter cycles per launch), so the deletion is worth something concrete.

10.4a 000 THE ORACLE OBJECTION IS ANSWERED — mmiotrace, and it is still in mainline

[w821] §10.3 argued the real cost of a no-GSP mode is **losing the oracle**, since openrm has no native Turing+ path and nouveau is only a register dictionary. [owner]:

“Nouveau has exact traces of what the proprietary driver does. Plus we can trace: we shadow- overwrite the MMIO map function in Linux so it all lands in a trap window, then we observe every trap and we know what it calls, what we must implement. If it’s only init it’s trivial.”

□□□ **That is CONFIG_MMIOTRACE, it is the technique nouveau was built with, and it is alive in mainline today.** Verified against a kernel tree cloned **2026-09-20**: arch/x86/mm/kmmio.c, mmio-mod.c, testmmiotrace.c, Documentation/trace/mmiotrace.rst — whose own opening reads “*built for reverse engineering any memory-mapped IO device with the Nouveau project as the first real user*”, linking to nouveau.freedesktop.org/wiki/MmioTrace.

⇒ □ **The objection does not stand.** We are not limited to nouveau’s *knowledge* — we can **generate the trace ourselves**, from the proprietary driver, on the exact chip we care about. □ And the two sources compose into a complete oracle pair that neither is alone:

source	gives
nouveau	□ semantics — what each register <i>is</i> , the field layouts, the meaning
mmiotrace	□ behaviour — what NVIDIA’s driver actually writes, in what order, on this die

⇒ [PROPOSE] **Adopt this as the standing method for the non-GSP plane**, exactly as the ogkm differential is the standing method for the GSP plane.

□ **Its documented limits, so nobody discovers them at 3 a.m.:** - **x86/x86_64 only** □ (our host is). - ∅ **It takes all but one CPU offline.** SMP tracing is unreliable and *silently* drops events. ⇒ We see a **uniprocessor** init, and concurrency is invisible. □ Check the lost-event counter **every run** — this tree’s own lesson about instruments that fail quietly applies directly. - ∅ It traps **ioremap’d kernel MMIO**. Userspace’s mmap of the doorbell page does **not** go through ioremap and will not appear. □ Irrelevant for init — which is the owner’s point (“*if it’s only init it’s trivial*”) — and a **hard limit** for anything else.

10.5 [PROPOSE] A spike, not a commitment — and the decisive hour

∅ Do not adopt this from a line count. □ **Run the smallest decisive experiment first:** diff nvkm/subdev/devinit/tu102.c against nvkm/subdev/devinit/r535.c. devinit is the *first* thing either path does, it is small, and the diff shows the shape of every other subdevice’s split.

⇒ If that diff reads as “a bounded register sequence we could answer”, the idea is live and the next question is the oracle gap in §10.3. If it reads as “arbitrary board bring-up”, it is answered, and the answer took an hour.

∅ **[SUPERSEDED w821]** This section previously ended “the prerequisite is still §1’s one-hour measurement; building a no-GSP mode for a configuration nobody has confirmed exists would be the most expensive way to answer a question one command settles.” □ **The measurement was taken and the configuration is real** (§1). The prerequisite is discharged.

10.6 □ OWNER RULING, w821 — the option stays open, and nouveau is the standing oracle

“Pre-Turing is still significant... So we get both guaranteed Windows and pre-Turing. Just ensure this remains open. And keep nouveau now as oracle for everything without GSP.”

ruling	consequence
□ The no-GSP mode stays OPEN	Not a spike to be closed out. It is carried as a live design option, and §§2–9 of this document are its requirements list
□□□ Pre-Turing is a GOAL, not scope creep	⇒ THE_ARCHITECTURE_v3.md §0.0.1 is amended : the Turing floor is architectural <i>for the GSP plane</i> , and a no-GSP plane is exactly what lifts it. Two planes, two floors
□ nouveau is the standing oracle for everything without GSP	Same status ogkm holds for the GSP plane. research_clones/nouveau-src (sparse checkout of drivers/gpu/drm/nouveau)
□ Mine ogkm for the leftovers	Whatever residue it holds on Windows behaviour or non-GSP behaviour gets recorded; the rest is nouveau
□□□□ GSP REMAINS THE PRIORITY TARGET	[owner] “it will be our more stable version as we have more source available.” ⇒ The no-GSP plane is additive reach, never a replacement

□□□ **And that ranking is exactly right, for the reason §10.3 gave — which was wrong as a blocker and right as a priority.** The two planes do not have equal evidence:

plane	oracle	strength
GSP	ogkm — actual NVIDIA source	□ tells us what the guest <i>will</i> send and what it does with the reply, before we build anything
no-GSP	nouveau (semantics) + mmiotrace (behaviour)	● tells us what <i>one driver</i> did on <i>one die</i> in <i>one uniprocessor boot</i>

⇒ ∅ A trace is a **sample**; source is a **specification**. mmiotrace closes the gap enough to make the no-GSP plane *tractable*, and not enough to make it *equally trustworthy*. □ Stability follows evidence, so GSP leads and the no-GSP plane follows it — and where the two planes share machinery (the whole of Part 1 §2: doorbells, channels, pushbuffers), **the GSP plane’s design is the one that sets the shape.**

□ **And one corroboration the owner supplies from the sibling project:** nvkvm-pv established that for the **userspace ioctl** surface, pre-Turing constants **are published in ogkm**, and it got pre-Turing working on a sub-branch. □ Scope that precisely: it establishes the **constants** exist, on the **Mode-1 ioctl** plane. It does **not** establish that per-chip *register-level HAL implementations* for pre-Turing are present — that is a separate question, and it is Task B of the ogkm residue survey now running.

10.7 The no-GSP plane’s boundaries and sources — owner rulings, w821

(a) ∅ **The no-GSP plane’s ceiling is AMPERE — and it is set by OUR ORACLE, not by NVIDIA** [owner] “and for Blackwell it’s a prerequisite.” □ True **on Linux**, and I first recorded the partition on that basis. ∅∅ **[CORRECTED w821 — right conclusion, wrong reason, and the right reason gives a better boundary.]**

Why the original reason does not hold. “GSP is required on Blackwell” follows from a chain that is **entirely about Linux packaging**: NVIDIA’s proprietary Linux module does not support Blackwell → only the open modules do → the open modules are GSP-only. ∅ None of that is a statement about the hardware, or about NVIDIA’s **closed Windows driver**, which plausibly retains monolithic RM for every chip it supports.

□□□ **And the real boundary is nouveau’s native engine coverage, which is checkable:**

die	native .gr	native .ce	no-GSP plane reachable?
pre-Turing → Turing (TU10x)	□ tu102_gr_new	□ tu102_ce_new	□ yes
Ampere (GA10x)	□ ga102_gr_new	□ ga102_ce_new	□ yes
Ada (AD10x)	∅ none	● reuses ga102_ce_new	∅ no — no native GR
Blackwell (GB20x)	∅ none	∅ not bound in the chipset entry	∅ no

⇒ □ **The partition stands, at a different line:**

plane	covers	bounded by
GSP (□ priority)	Turing → Blackwell	— the only plane above Ampere
no-GSP	pre-Turing → Ampere	∅ nouveau’s native GR/CE , not NVIDIA

□□□□ **And this is the strategically good news, which the packaging argument obscured: the no-GSP plane is BOUNDED AND WILL NEVER GROW.** nouveau moved to GSP for new hardware precisely because native reverse engineering stopped being tractable — signed firmware, complexity. ⇒ The plane covers hardware that **already exists and will not expand upward**. That makes it a **finite, closing investment**, categorically different from a plane requiring work every generation. □ It also means the ceiling rises only if nouveau gains native Ada/Blackwell GR, which should be treated as **not going to happen**.

□ **And the tension I flagged earlier dissolves.** The RTX 5090 report of *disabling* GSP on Windows is perfectly consistent — NVIDIA’s closed driver can do that. ⇒ It is simply a **configuration we cannot serve**, for want of an oracle, not one that contradicts the record.

∅ **The prediction I recorded is withdrawn.** I wrote that “Windows must default GSP ON for Blackwell SKUs, since a driver cannot ship a default that cannot work.” □ That inherited the Linux-packaging error. [forum, w821, weak] two posters on the thread the owner linked assert the opposite — “It’s not enabled on anything by default” and that 50-series owners were reporting results only **after** setting the key. ∅ Still no `nvidia-smi` before-state on Blackwell, so this is **weak corroboration, not evidence** — but it points away from my prediction, and the prediction had no mechanism behind it.

(b) □ Replay fidelity is relaxed, because most of the delta is stub-able [owner] “anything the driver does extra and not on GSP is mostly stub, so replay is less of an issue.” □ Corroborated by §10.2’s measurement: **~22 400 of the ~67 000 subdev/ lines are board and power management** — `clk`, `bios`, `i2c`, `therm`, `volt`, `gpio`, `mxm`. An **emulated** GPU has no clocks to raise, no fans, no thermal sensors and no I²C bus. ⇒ Those answer *plausibly* rather than *faithfully*, which is a far weaker obligation than replay.

□ **The boundary to hold: stub means answer without modelling, not ignore.** A register the driver **polls for a state transition** is not stub-able however cosmetic it looks — this tree has already paid for that once with a `TRIGGER` bit that must clear or the guest hangs.

(c) □□□ Deriving from nouveau is LICENSED — but derive from `rnndb`, not from its C headers [owner] “we need to avoid that we maintain stuff per die, so in that case derive from nouveau headers is allowed. We can use nouveau — it’s tested on a wide variety of GPUs.” □ The breadth is real: **131 chip entries** in nouveau’s device table, spanning `nv04` → Blackwell.

⇒ **[PROPOSE] Take the licence, but take it one level better.** The nouveau ecosystem publishes `rnndb` — “an *rnn* database of nvidia MMIO registers, FIFO methods, and memory structures” in the **rules-ng-ng XML** format, with `rnn` providing tools and libraries for it. `demmio` “annotates known registers based on its database of registers”, so it is demonstrably machine-consumable.

□ Why XML beats nouveau’s C headers here, and it is this document’s own rule: §0.3 rule 2 forbids parsing C with regex and requires a real parser. **rnndb is already structured data with a schema** — deriving from it satisfies that rule *natively* instead of needing a C front end. □ **[UNVERIFIED]** `rnndb`’s exact per-chip coverage and whether `rnn` ships a header generator — the README states neither. **Check before committing to it.**

∅ **And a provenance rule that must travel with the licence.** ogkm constants are **vendor truth**; rnndb/nouveau constants are **reverse-engineered**, possibly incomplete, and use *nouveau*'s names rather than NVIDIA's. ⇒ Generated code must **mark which source a constant came from**, and where both exist, **ogkm wins**. A generator that silently blends the two produces a table nobody can audit — the failure this tree already named for capture-derived tables.

(d) ∅ There is NO published corpus of nouveau MMIO traces [owner] *"if nouveau leaked MMIO traces for many GPUs it's a perfect oracle."* ∅ **Checked: it does not exist publicly.** The nouveau wiki asks contributors to **send** dumps privately — *"send them to mmio dot dumps at gmail dot com... the archive file name should contain the PCI id and GPU family"* — and no public archive was found. What is public is the **dictionary** (rnndb) and the **tool** (demmio), not the traces.

⇒ □ **So we generate our own, which is what §10.4a already proposed.** And the need is narrower than nouveau's: we do not want traces of 131 chips, we want traces of **the dies we ship**, doing **init**, from **NVIDIA's driver**. ⇒ nouveau supplies breadth of *semantics*; our own mmiotrace runs supply depth of *behaviour* on the dies that matter.

Sources: [envytools](#) · [nouveau MmioTrace wiki](#) · [kernel mmiotrace docs](#)

Part 6 — What is still open

STATUS: LIVE, 2026-09-20 (w821). Everything the design does **not** decide, with what each would cost to close.
∅ Nothing here is a placeholder for work in progress — these are the places a reader should push, because they are the places I would.

1. Open, and cheap to close

#	question	what closes it	cost
1	Does the page-table walker survive a hardened sandbox? The GPU-side walker runs in-process; under a sandbox profile that denies process spawning, the device nodes must already exist. That is a host prerequisite either way — but it is unmeasured	one CI test: initialise the driver API and run one launch under exactly the sandbox string a production manager uses	an afternoon
2	Is the doorbell-table scan really sub-microsecond? The decision to delete the hint ring rests on it, and the number is arithmetic	a microbenchmark on the bench box, warm and cold	an hour
3	Do two clients mapping the same usermode object land on one guest-physical page? This decides whether “ <i>one doorbell page</i> ” is literally true on Hopper+	read the mapping path, or observe two clients in a guest	a morning
4	Can both the BAR0 and BAR1 forms of the usermode object be live at once in one guest? Both are legal on Hopper+	the same read	with #3

2. Open, and genuinely undecided

#	question	why it is hard
5	Windows beyond consumer Turing. The default is per-SKU by construction — the policy takes device and subsystem id, a workstation/server flag, and the driver mode. We measured one corner	the deciding logic is stripped from the published source. ⇒ It is a measurement programme , not a reading exercise: a Windows box per SKU class
6	The hand-maintained overlay. Read/write-only-ness generates from the published access codes; write-1-to-clear, trigger and data-port semantics do not — the pending register and its enable port carry the <i>same</i> code	~a dozen names per family, hand-written, and the build must fail when a register has no entry . □ It is small, and it is the one place a per-die fact is maintained by hand rather than derived

#	question	why it is hard
7	The line target. ~50 k hand-written is reachable only if generation holds and the served-control set stays near 60; an adversarial re-count puts the all-in figure at 70-90 k	∅ These are different denominators, not competing claims. The number that settles it is whether a second driver version adds descriptor rows or code

3. Deliberately not decided — deferred with a reason

#	item	why it is deferred, not open
8	The no-GSP plane (a guest whose firmware is disabled — which is the default on consumer Windows)	it has a different oracle, a different floor, and a bounded ceiling at Ampere. □ It is additive reach , and letting it shape the trap path or the chip descriptors now would make both worse
9	Display	NVIDIA ships a displayless class; it is a later phase and not on the path to compute
10	Peer-to-peer between GPUs	a third leaf verb, fully specified in the design, built after single-GPU works
11	Interrupt delivery	deferred only because the things that need it poll , and only on condition that a blocking-sync arm enters the suite so the gap is red rather than silent

4. ∅ Decided the other way — what we give up, and why it is worth it

□ These are not gaps. They are refusals, and a reader should attack them as choices.

we refuse	we give up	why that is right
A guest IOMMU	guests configured with one; eventually Windows DMA protection	every guest address we consume resolves through a guest-physical-keyed layout, so a guest IOMMU is bypassed by construction . Supporting it means a translation layer on every consumer including a GPU kernel that cannot call the VMM . No CUDA or LLM guest needs one
Broadcast device groups	multi-GPU SLI-style configurations	it is the only case that produces “ <i>one VA space, many page-directory bases</i> ”, and CUDA never uses it. Refusing it removes a whole class from the address model
Fault recovery (we deliver faults; we never recover)	nothing a guest can reach	the recovery registers are host-kernel-owned and unimplementable from userspace . = The host driver recovers our twin and we tell the guest through the mechanism it already has
Guest-chosen pointers in host calls	the memory class that carries one	it would hand the host driver an address the guest chose. □ Our host verbs take no guest flag word either — the same rule, applied to bits instead of pointers

we refuse	we give up	why that is right
Emitting the firmware bug-check event	nothing	it crashes the host OS on Windows. We are the firmware. Any path that could induce us to emit it is a guest-crash primitive
Pre-Turing, on this plane	older hardware	there is no firmware processor to impersonate below Turing. ∅ Not a scope choice — the architecture does not exist there. The no-GSP plane is what would reach it

5. □ The three things most likely to be underestimated

Not open questions — **predictions**, recorded so they can be checked later.

1. **Teardown and re-init.** A guest crash, a forced reboot, a kexec and a machine reset never run the driver's unload path, and the stock driver **refuses to boot while the protected firmware region is still up**. ⇒ Every gate should be run **twice in one process**. A plane that works on the first init and not the second does not work.
2. **Error propagation.** Named refusals are load-bearing — but what the *guest* does with each one is modelled nowhere, and NOT_SUPPORTED is a status the driver **forgives**. ⇒ A wrong refusal does not fail; it runs on.
3. **The served-control set.** It is the budget's weak point and the surface most likely to grow under contact with a real workload.